KANT

팀별 로봇 미로 주행 실습

Isaac Sim · Isaac Lab 강화학습

팀별 URDF·USD로 실습 / 시연: M38

학습 목표와 팀별 적용

각 팀은 자기 로봇의 URDF와 USD를 준비한다.
관절·좌표·물성 정보를 설명하고 차이를 검증한다.

로봇 구조에 맞는 제어기와 센서 구성을 선택한다.
설정, 로그, 캡처로 결과를 재현한다.

M38은 시연 사례다. 관절명·치수·구동 방식을
다른 팀 로봇에 그대로 적용하지 않는다.

공통 개발 환경과 설치 확인

Ubuntu 24.04 / ROS 2 Lyrical / Isaac Sim 6.1.0.0
실습 버전을 고정하고 각 팀 PC 상태를 기록한다.

cat /etc/os-release
nvidia-smi
df -h .
GPU·메모리·설치 경로를 팀별로 확인한다.
Lyrical의 Sim native ROS 연결은 별도 검증이 필요하다.

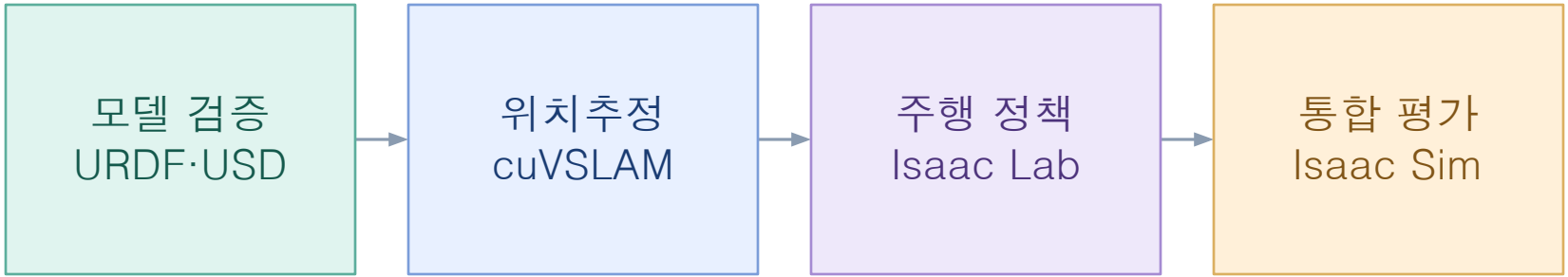
설치 환경 분리

python3 -m venv ~/isaacsim6_venv
source ~/isaacsim6_venv/bin/activate

Sim 설치는 버전을 고정한 setup_sim.sh를 사용한다.
ROS 패키지와 Python venv의 역할을 구분한다.

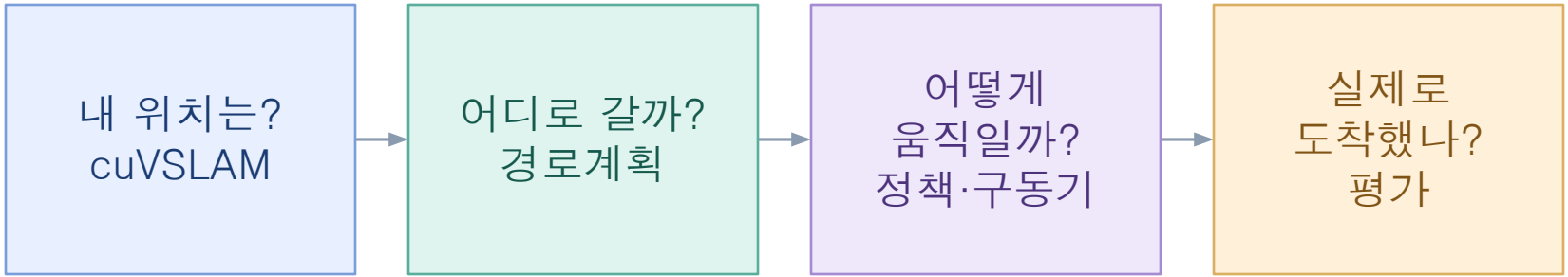
설치 로그와 package version을 저장한다.
세부 명령과 실행 확인: TEAM_LAB.md

자율주행 실습의 네 단계



센서·위치·행동·물리 결과를 단계별로 확인한다.

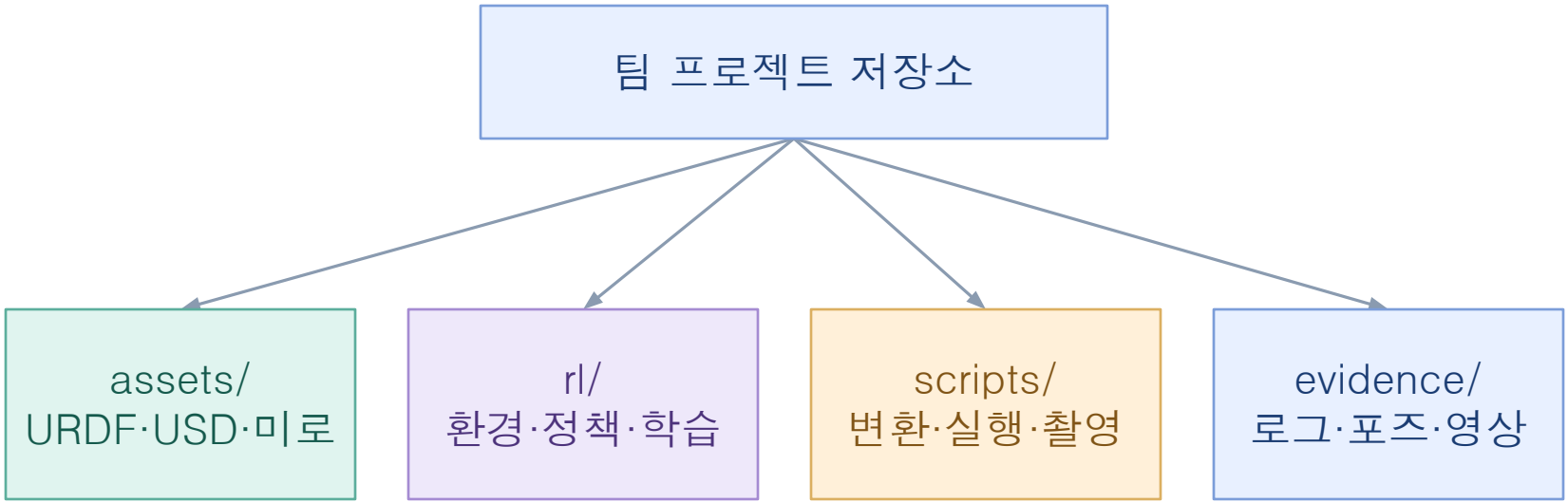
위치추정과 자율주행의 차이



위치추정만 실행해도 자율주행이 완성되는 것은 아니다.

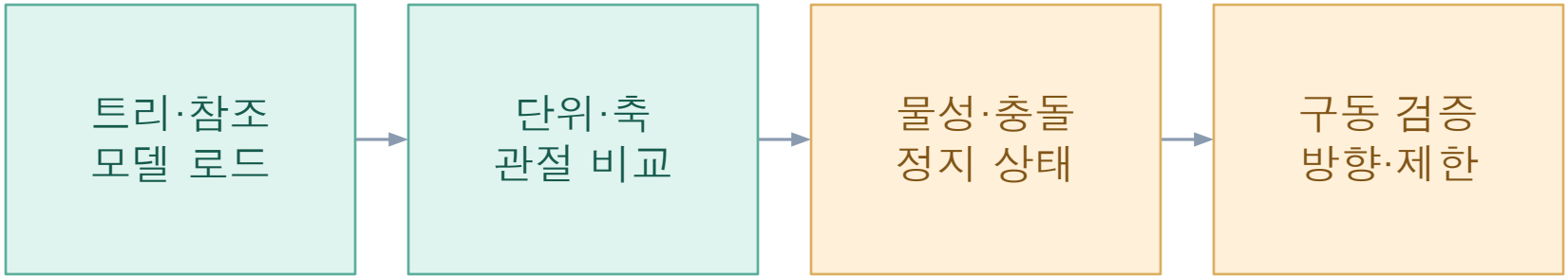
KANT

팀별 프로젝트 구조



모델·학습 코드·도구·증거를 서로 다른 폴더에 둔다.

URDF와 USD 모델 검증



겉모습이 정상이어도 articulation root와 충돌 설정은 별도 확인한다.

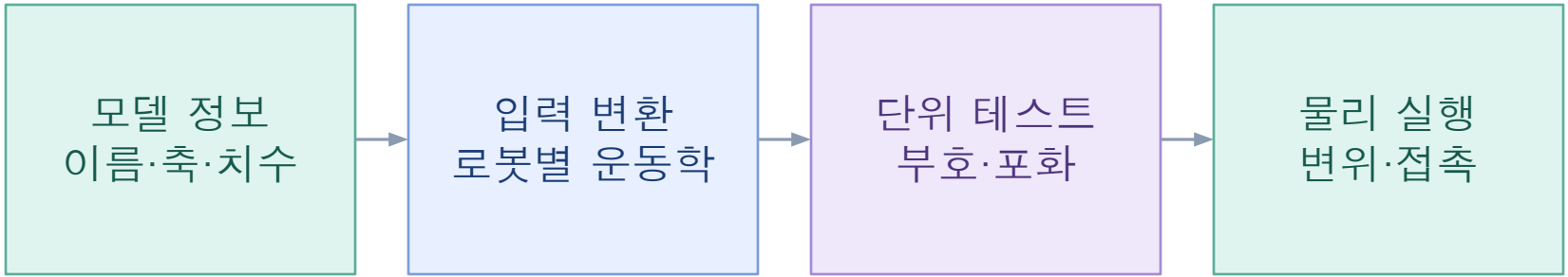
로봇 구조에 맞는 제어기

차동구동: 좌우 바퀴 속도
스워브: 각 모듈 조향각과 바퀴 속도

팔 로봇: 관절 위치·속도 또는 궤적
그 밖의 구조: 해당 기구학과 구동 인터페이스 정의

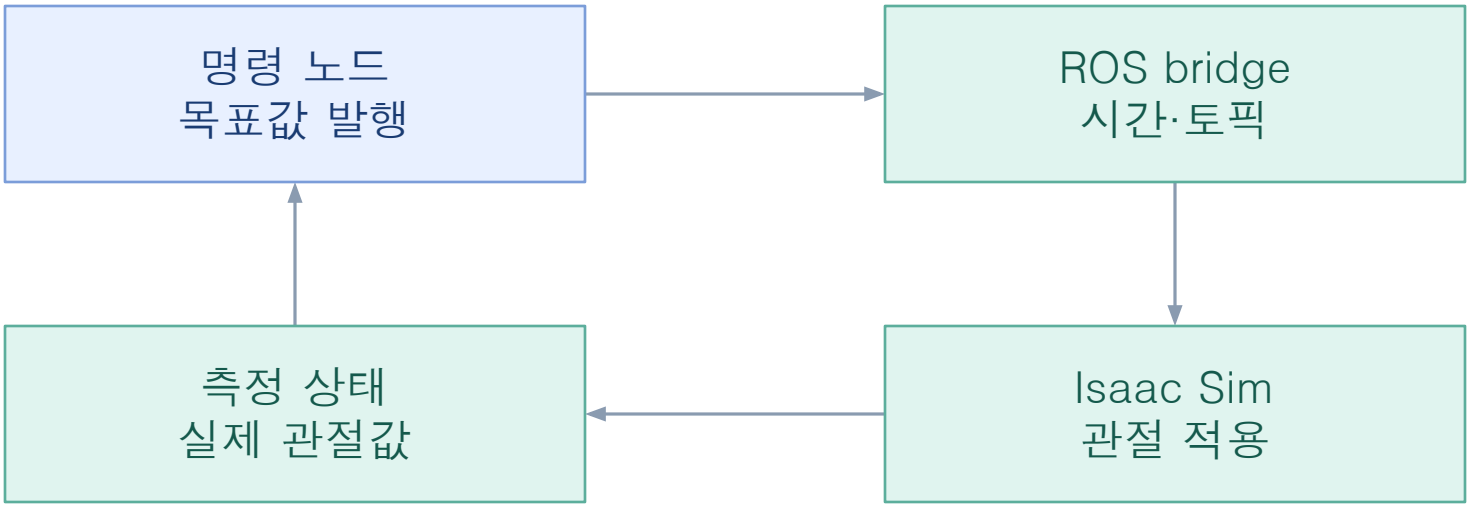
시연 M38 예제는 스워브 구동이다.
앞뒤 바퀴 축 방향과 회전 부호가 서로 다르다.

코드 작성과 검증 순서



시연의 스워브 운동학은 다른 구조의 로봇에 맞게 바꾼다.

ROS 연결과 시간·좌표



timestamp, TF, 관절명을 맞추고 수신 만료 시 정지 동작을 검증한다.

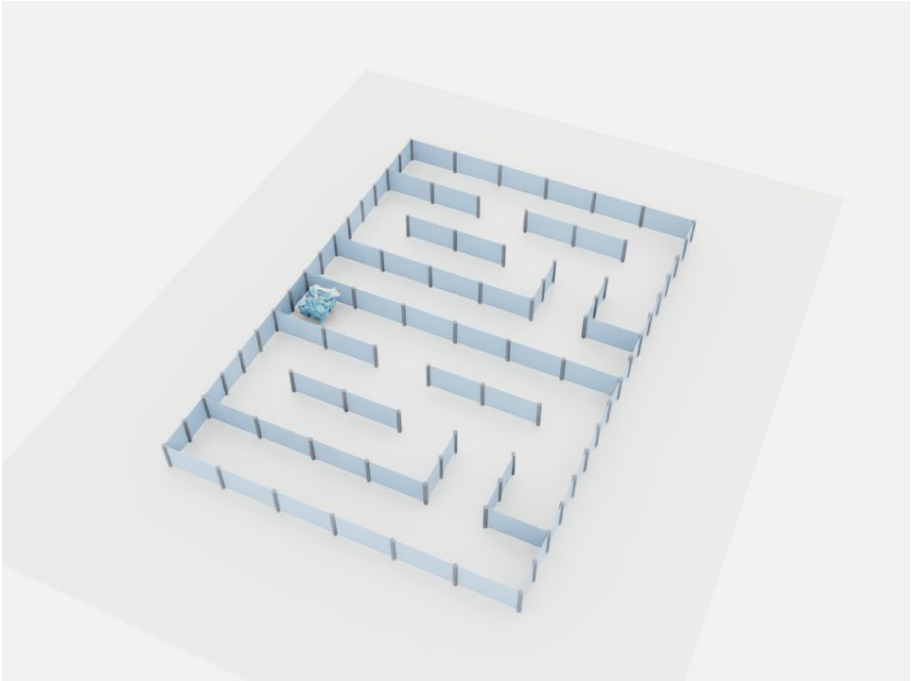
빌드와 실행 검증

source /opt/ros/lyrical/setup.bash
colcon build --packages-select <팀_패키지>

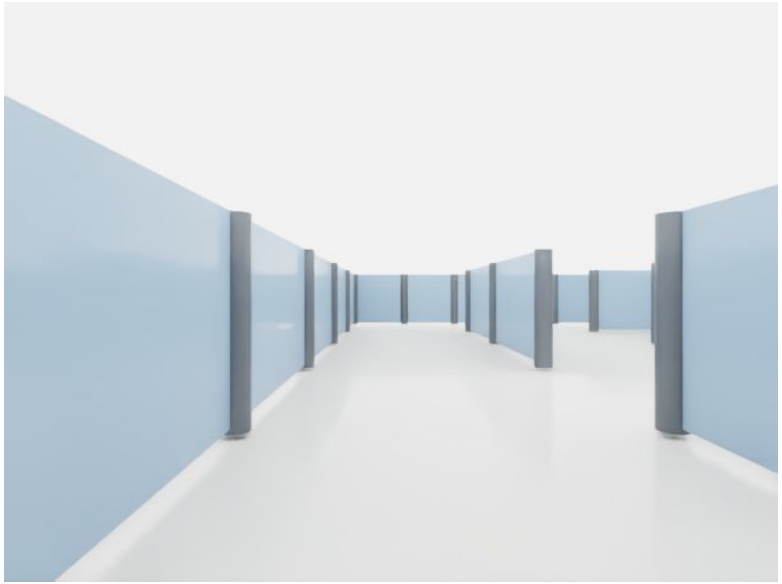
source install/setup.bash
ros2 launch <팀_패키지> <팀_launch.py>

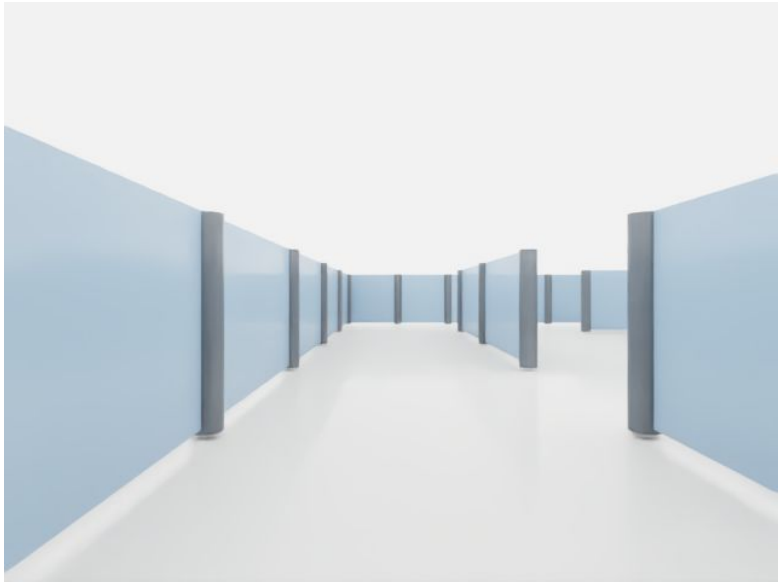
노드 로드, 명령 전달, 측정 상태를 차례로 확인한다.
M38 실행 예제는 시연용 LAB_GUIDE.md에 분리했다.

시연: 제공 미로와 로봇



시연: 미로의 좌우 카메라 영상





640 × 480, 가상 기준선 0.05 m / 정적 렌더이며 이동 SLAM 검증과 구분한다.

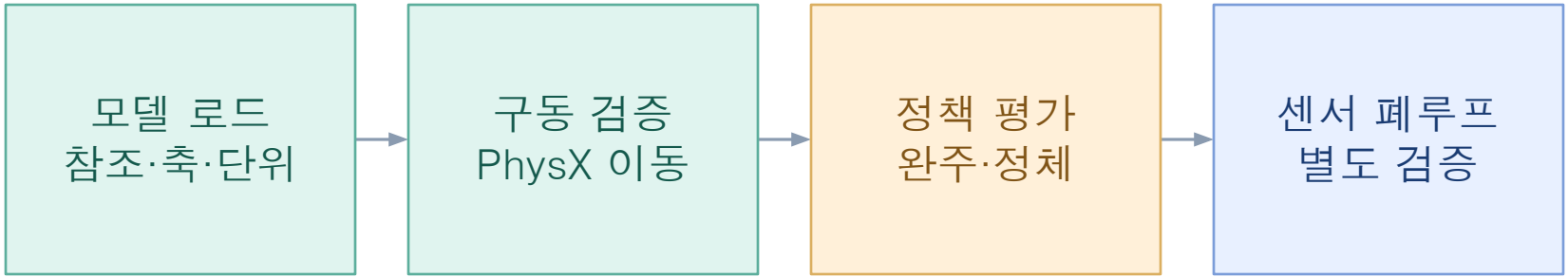
센서와 알고리즘 선택

스테레오 영상: CameraInfo·좌우 동기화·optical TF
LiDAR: LaserScan·거리/각도 규약·scan TF

cuVSLAM과 slam_toolbox·AMCL은 입력이 다르다.
팀 목표에 맞는 파이프라인을 구성한다.

깊이 예제: $Z = fx \times B / disparity$
fx와 기준선 B는 각 팀 카메라 값으로 바꾼다.

단계별 완료 기준



현재 물리 이동과 PPO 학습·평가를 실행했다. 상세 결과는 뒤의 평가 자료에 표시한다.

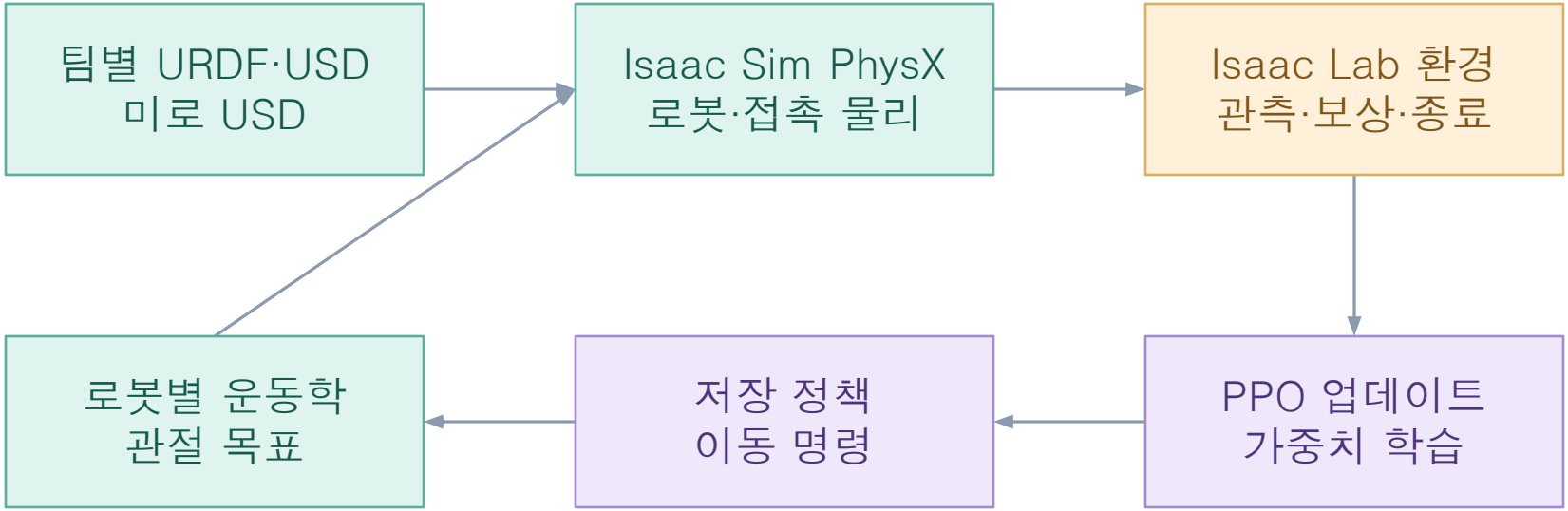
기초 실습 체크포인트

설치법 / 파일 구조 / 제어·센서 원리 / 코드 설명

설정 파일 / 재현 명령 / 테스트 로그
자기 로봇의 실행 PNG와 짧은 GIF

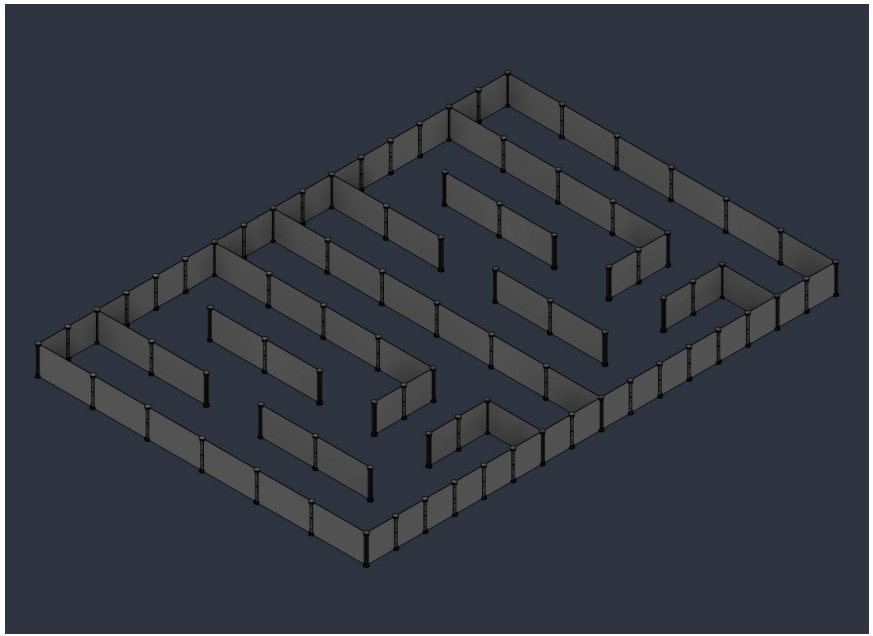
가정값과 실패 조건, 미완료 항목을 명시한다.
다른 팀이 자료만 보고 같은 결과를 재현하는지 확인한다.

미로 주행 프로젝트 전체 구조



실선: 현재 물리·학습 경로 / 센서 위치추정 연결은 별도 확장 단계

제공 미로의 치수와 시뮬레이션

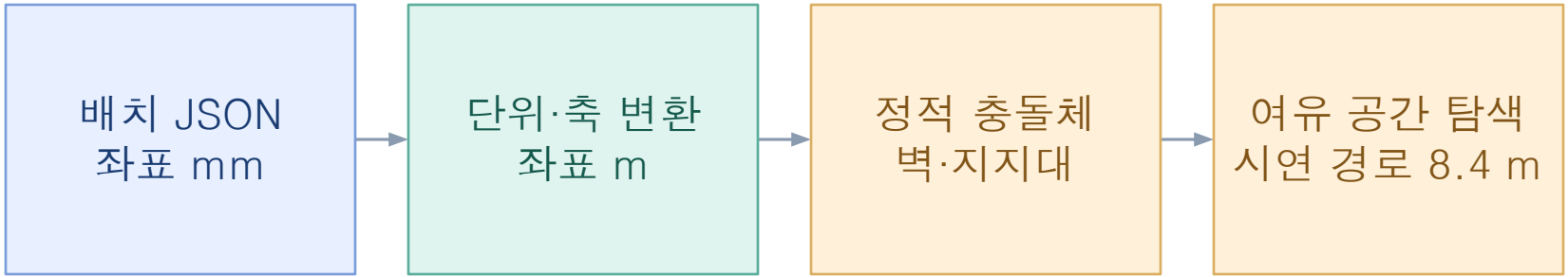


바닥 4.464 × 6.364 m
높이 0.39 m

벽 80개와 지지대 83개

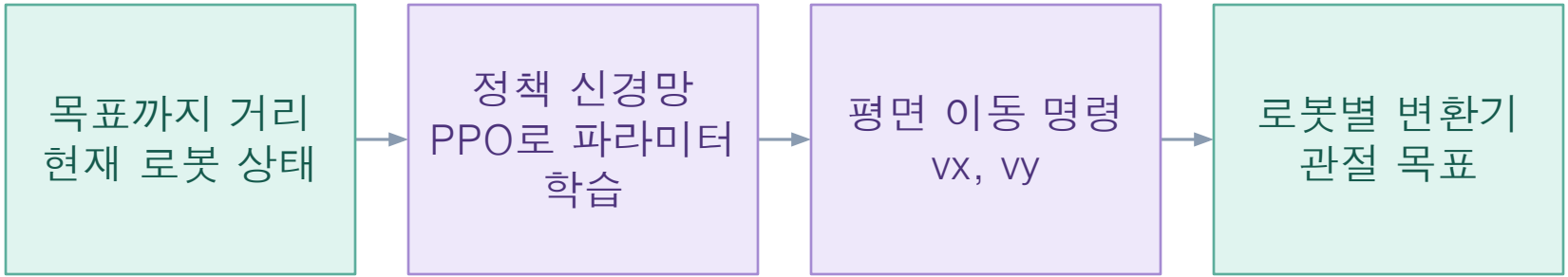
벽 패널은 치수를 유지하고
지지대는 지름 60 mm로
단순화

미로 생성과 경로 준비



경로 준비 단계는 알려진 지도를 사용한다. 미지의 미로 탐험은 추가 과제다.

강화학습이 결정하는 값



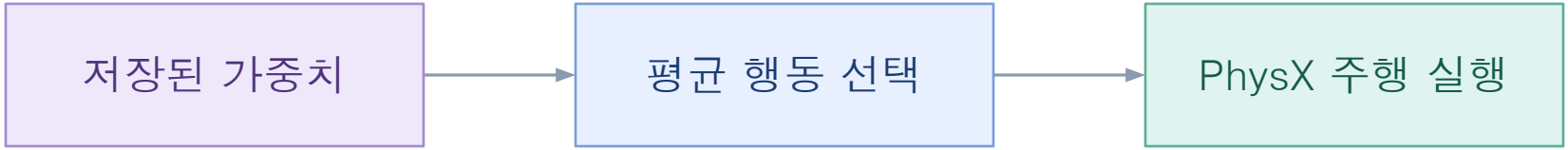
시연 정책 출력은 2개 값이다. 모터 구동 구조는 각 팀 로봇에 맞게 작성한다.

학습과 실행의 차이

학습



평가



평가 중에는 가중치를 변경하지 않는다.

환경 설치와 버전 고정

```
python3 -m venv ~/isaacsim6_venv

~/isaacsim6_venv/bin/python -m pip install --no-cache-dir ₩
  "isaacsim[all,extscache]==6.1.0.0" ₩
  --extra-index-url https://pypi.nvidia.com

nvidia-smi
~/isaacsim6_venv/bin/python -m pip show isaacsim
```

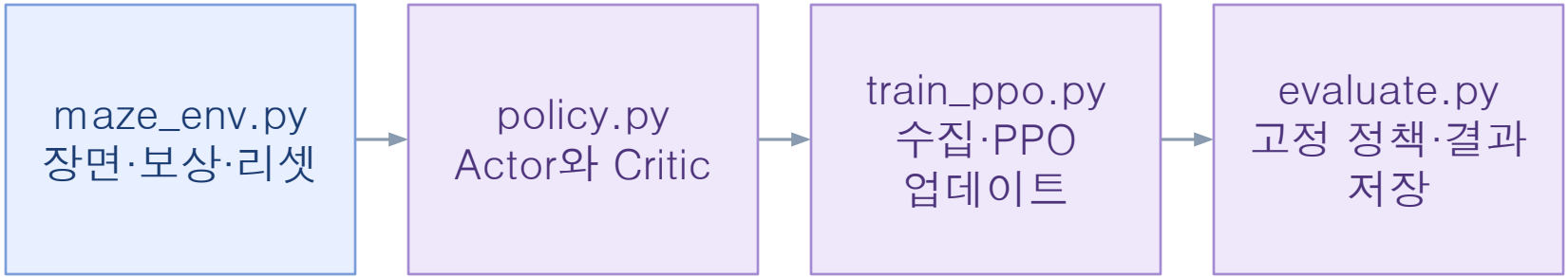
Isaac Lab 실행 환경

수업에서 지정한 Isaac Lab 소스 압축파일을 내려받는다.

압축을 풀어 ~/IsaacLab에 배치한다.
pyproject.toml과 uv.lock이 있는지 확인한다.

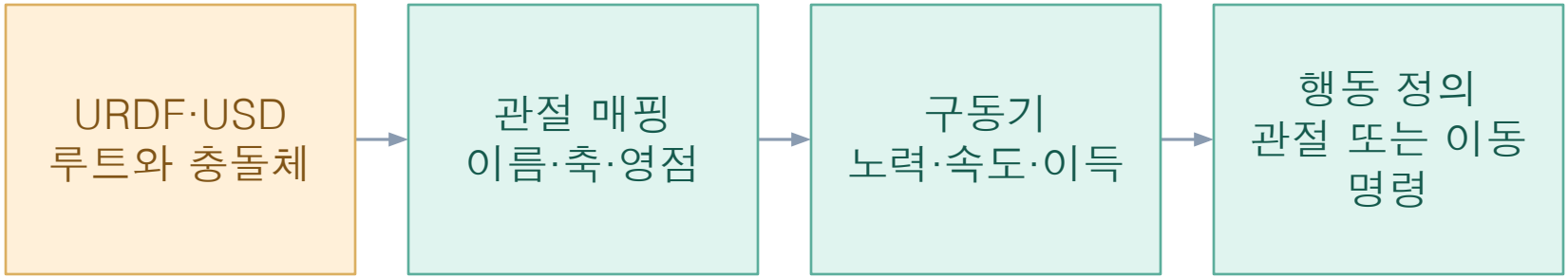
cd ~/IsaacLab
uv sync --frozen
cd <프로젝트_폴더>

학습 코드 파일 구조



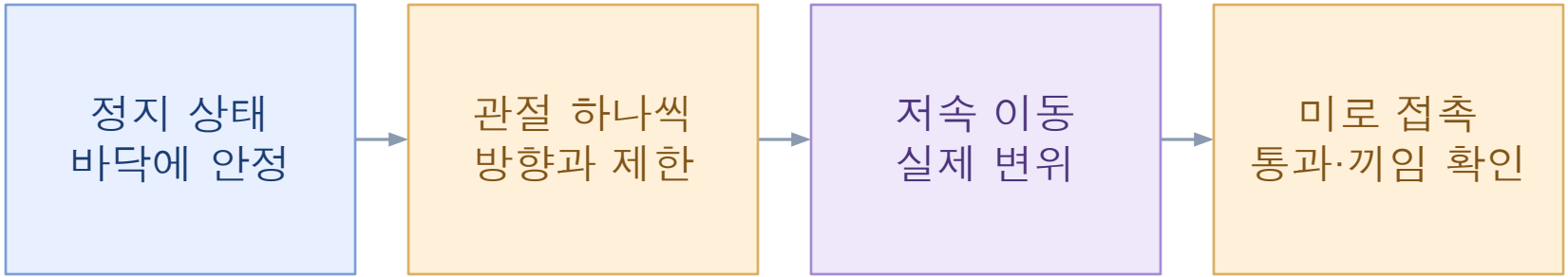
모델, 환경, 알고리즘, 평가를 분리하면 오류 위치를 좁힐 수 있다.

팀별 로봇 적용 항목



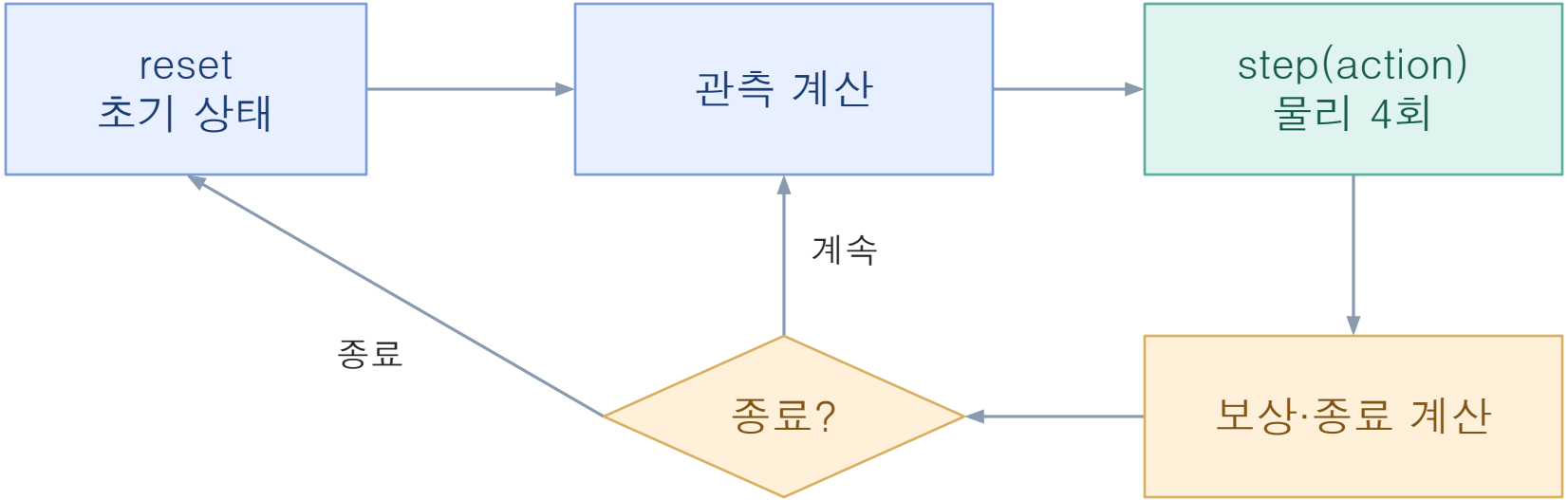
학습 횟수를 늘리기 전에 로봇이 해당 행동을 물리적으로 실행할 수 있는지 확인한다.

학습 전 물리 검증



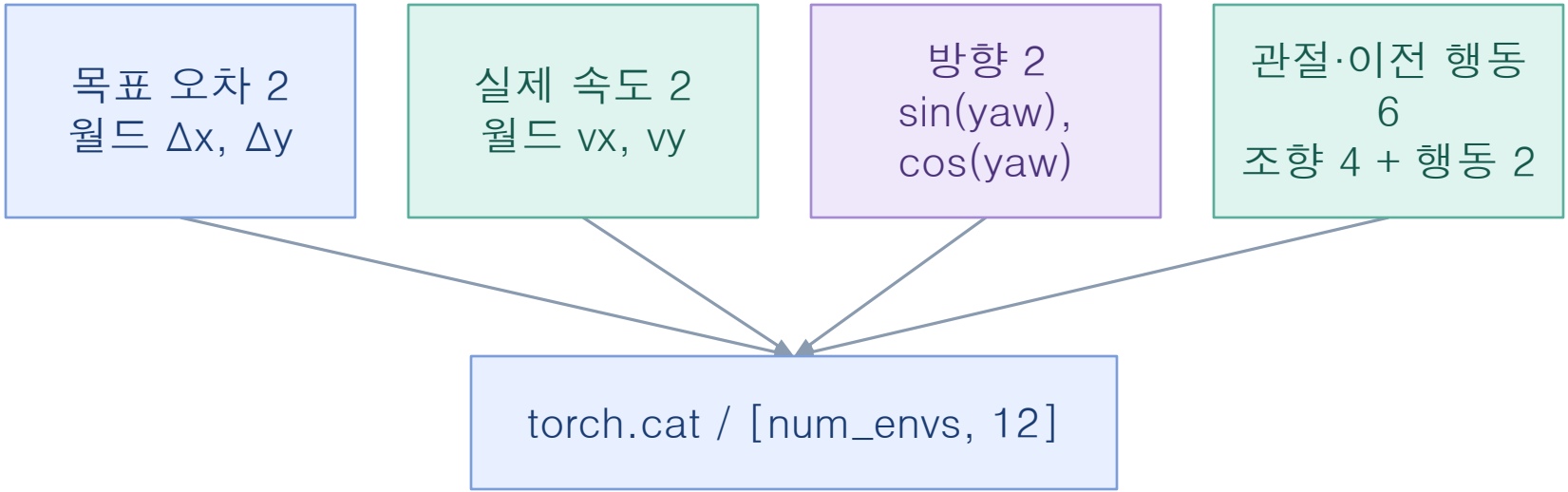
실행 로그: lab-physics-probe-root.log, lab-maze-smoke.log

환경 클래스의 실행 순서



physics dt = 1/60 s, decimation = 4, policy step = 1/15 s

관측 벡터 12차원



네 종류의 정보를 같은 순서로 연결해 정책에 입력한다.

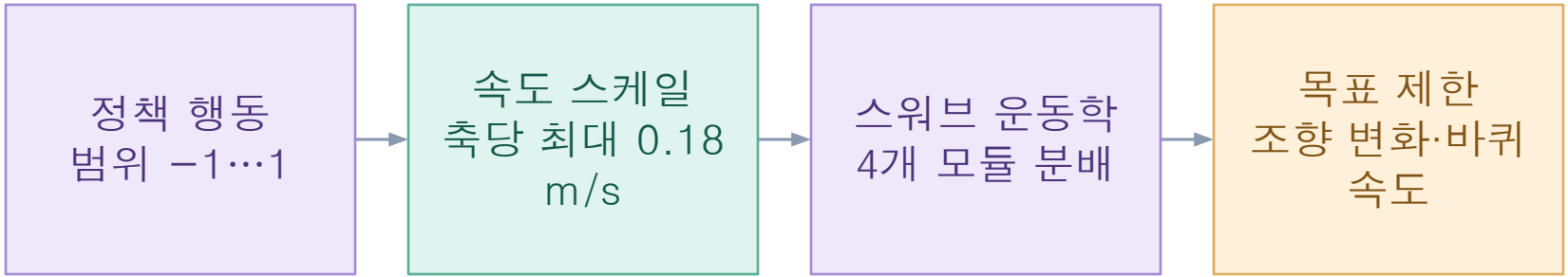
관측 코드 작성

```
delta = (self.goal - self.xy()).clamp(-1, 1)

obs = torch.cat([delta, vel[:, :2],
    yaw.sin()[:, None], yaw.cos()[:, None],
    steer_pos / (math.pi / 2), self.actions], dim=1)

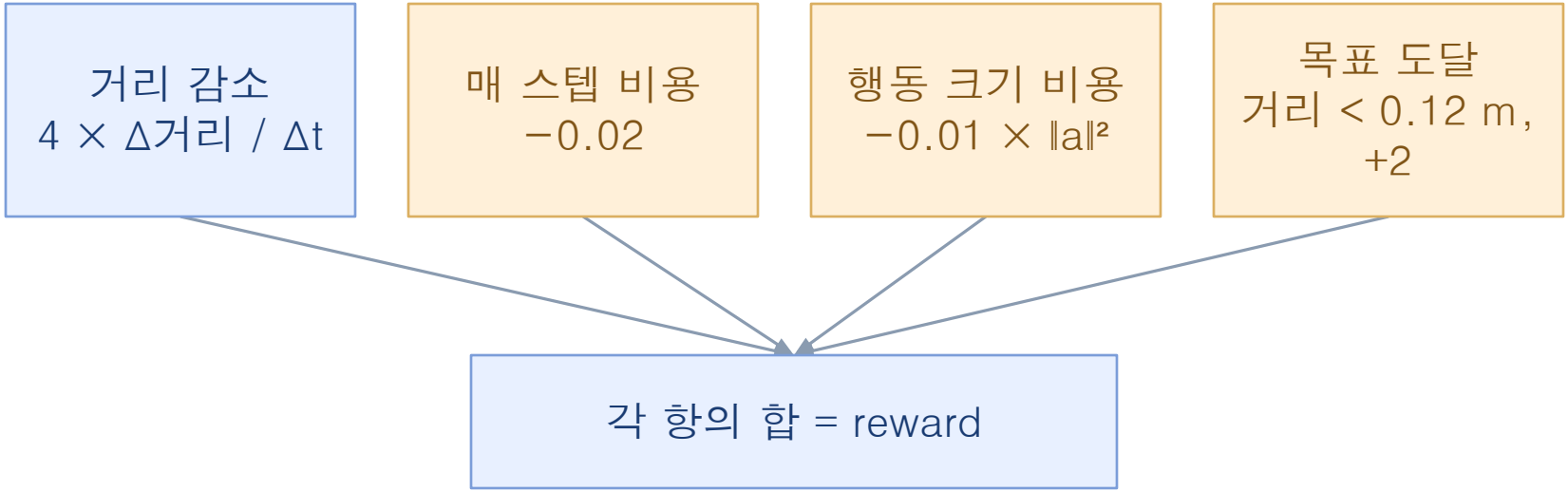
return {'policy': obs}  # [num_envs, 12]
```

행동과 관절 명령



정책 명령, 목표 관절값, 실제 관절값을 별도로 기록한다.

보상 함수의 구성



항별 크기와 실제 주행 결과를 함께 비교한다.

보상함수: 현재 기준식

$$r_t = 4\,\frac{d_{t-1} - d_t}{\Delta t} - 0.02 - 0.01\,\|a_t\|_2^2$$

$$+\ 2\,\mathbf{1}[d_t < 0.12]$$

진행 계수 4 / 시간 비용 0.02 / 행동 비용 0.01 / 도달 보너스 2

보상함수: 확장 설계 예시

$$r_t = w_p \Delta s_t - w_t \Delta t - w_a \|a_t\|^2 \Delta t$$

$$- w_s \|a_t - a_{t-1}\|^2 - w_c \, \mathbf{1}[\text{collision}]$$

$$+ w_g \, \mathbf{1}[\text{success}] - w_f \, \mathbf{1}[\text{failure}]$$

Δs: 경로 진행량 / Δt: 시간 / w: 조정 계수 / 이벤트 항은 한 번만 지급

보상 사례 ① 진행·시간·도착

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
목표로 접근	진행 계수 4	2·4·8 비교	목표 거리 감소
제자리 정지	시간 비용 0.02	0.01·0.02·0.04	정체·충돌 함께
과한 행동 억제	행동 비용 0.01	0.005·0.01·0.02	도착 시간·포화
목표 도착	도달 보너스 2	1·2·4 비교	완주율·반복 수령

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

KANT

보상 사례 ② 값보다 항의 크기

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
1 cm 전진	dt=1/15초	진행 항 +0.60	방향·단위 검산
1 cm 후퇴	같은 목표	진행 항 −0.60	후진을 항상 벌하지 않기
행동 두 축 최대	각 축 $\|a\| \leq 1$	행동 항 −0.02	진행 항과 비율
8초 정지	120 step	시간 합 −2.4	자살 종료 유인

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

보상 사례 ③ 벽 접촉과 거리

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
벽에 계속 밀기	충돌 이벤트 비용	작은 값부터 증가	접촉률·정지율
벽에 너무 근접	거리 여유 비용	안전거리 안에서만	최소 외곽 거리
바닥 접촉 벌점	접촉 필터 오류	벽·차체만 구분	서 있어도 벌점?
좁은 통로 회피	과한 안전거리	통로 반폭 이하 검토	통과 가능성

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

보상 사례 ④ 코너·후진·우회

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
직선 거리만 감소	벽 뒤 목표	경로 진행률 사용	벽 관통 지름길
코너에서 정체	먼 lookahead	6점 → 3점 비교	코너 여유·진동
막다른 길 탈출	후진을 일괄 벌점	목표 진행으로 보상	후진 회복 성공
우회 중 거리 증가	직선 거리 페널티	경로 잔여길이 고려	우회 경로 선택

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

KANT

보상 사례 ⑤ 진동과 구동 안정

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
좌우 명령 진동	행동 변화 비용	‖ a−a_prev ‖ ² 추가	전환 횟수·지연
조향각 반복 반전	±90° 분기	구동 분기부터 수정	바퀴 부호·정체
급가속·미끄러짐	가속도 비용	낮은 계수부터	미끄럼·주행 시간
전복·지면 이탈	기울기 + 실패	실패 종료·비용	안정 높이·완주

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

보상 사례 ⑥ 도착 판정과 종료

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
도착 직전 진동	반경 0.12 m	0.08·0.12·0.16	오차·도달 시간
목표 지나쳐 성공	거리만 사용	속도 조건도 추가	정지 후 도착
목표 보너스 반복	매 step +2	최초 도달·종료	보너스 횟수
넘어져 빨리 끝냄	실패 비용 없음	실패 비용·모델 점검	실패율·return

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

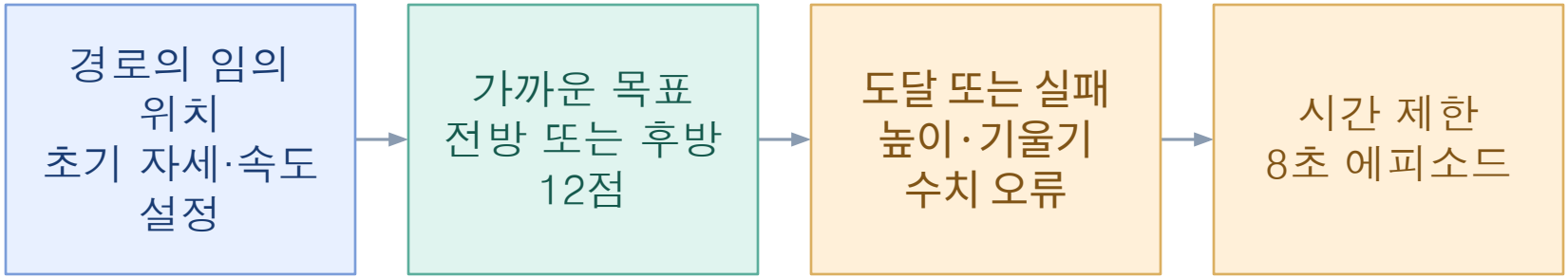
KANT

보상 사례 ⑦ 입력·분포·악용

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
접촉 벌점만 추가	벽 입력 없음	거리·접촉 관측 추가	상태 구분 가능?
한 방향만 성공	전방 목표만 학습	양방향·여러 시작점	시작점별 성공률
보상만 급상승	reset 거리 튐	기준 거리 재설정	reset 직전·직후
Sim만 잘 움직임	정답 pose 학습	노이즈·지연 단계화	추정 pose 평가

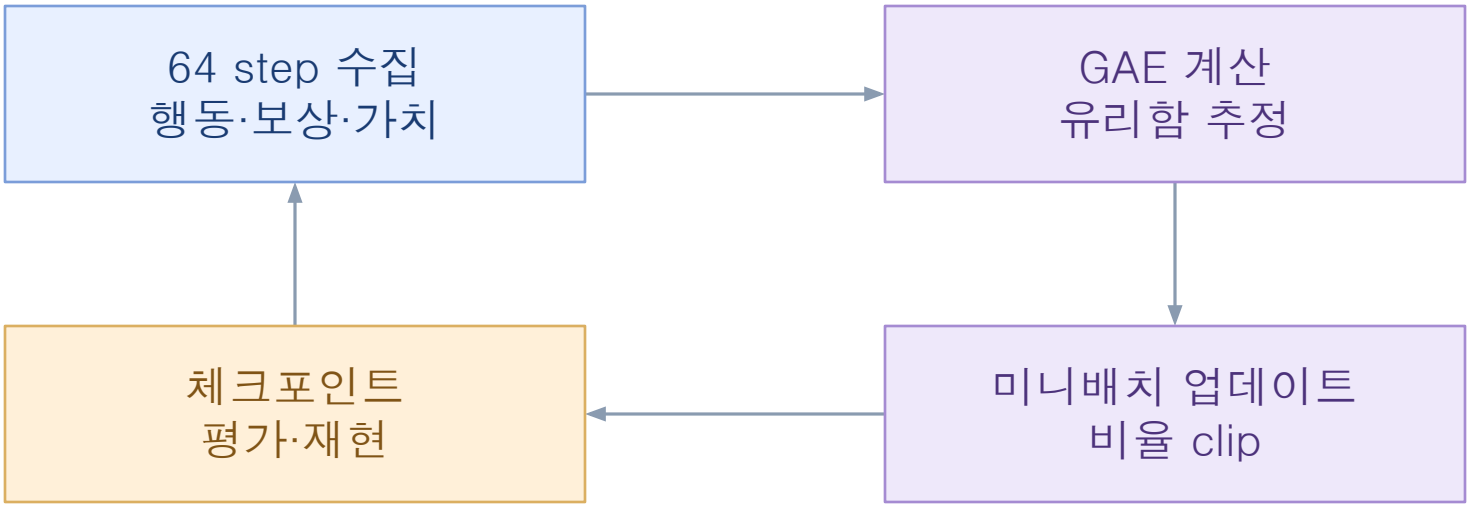
제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

리셋과 종료 조건



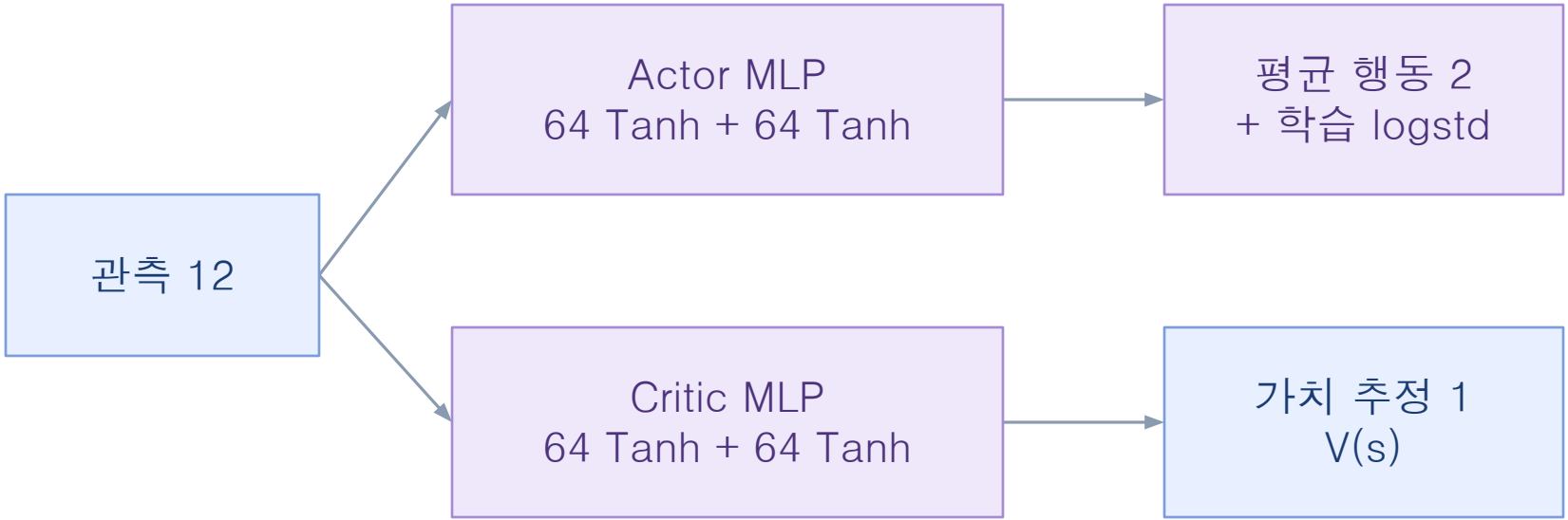
리셋은 학습 에피소드 경계에서만 실행한다. 평가 주행 중 위치를 옮기지 않는다.

PPO 학습 루프



rollout 수집에는 no_grad, PPO 업데이트에는 역전파를 사용한다.

Actor와 Critic 네트워크



Actor와 Critic은 각각 별도의 가중치를 학습한다.

PPO: 정책 갱신 목적함수

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

$$L^{\text{CLIP}} = \mathbb{E}_t\left[\min\left(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t\right)\right]$$

ρ: 새 정책과 이전 정책의 확률비 / Â: 유리함 / ε: 갱신 폭 조절

PPO: 실제로 최소화하는 손실

$$\mathcal{L} = -L^{\text{CLIP}} + c_v\,\mathbb{E}[(V_\theta - \hat{R})^2] - c_e\,\mathbb{E}[\mathcal{H}(\pi_\theta)]$$

$$\varepsilon = 0.2, \qquad c_v = 0.5, \qquad c_e = 0.005$$

정책 개선 + 가치 예측 – 탐색 보너스 / 계수 변경은 표의 증상과 함께 판단

PPO: GAE와 가치 목표

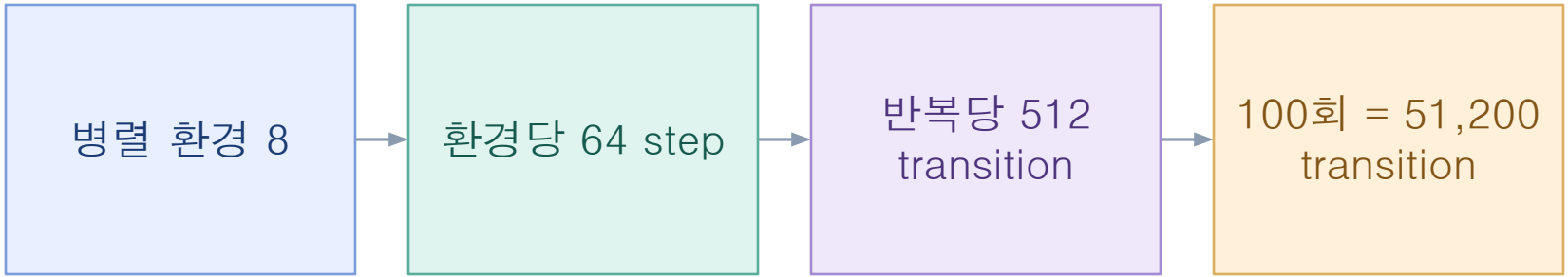
$$\delta_t = r_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t)$$

$$\hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1}$$

$$\hat{R}_t = \hat{A}_t + V(s_t), \qquad \gamma = 0.99,\ \lambda = 0.95$$

이 식의 d는 종료 마스크 / timeout은 별도 bootstrap 처리 필요

실행 파라미터와 샘플 수



비교 기준에 transition 수, 실제 경과 시간, 환경 수를 함께 적는다.

PPO 조정 ① 업데이트 크기

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
학습률	3e−4	불안정: 1e−4 비교	KL·성능 급락
clip epsilon	0.2	큰 갱신: 0.1 비교	clip fraction
update epoch	4	과적합: 2로 감소	KL·새 평가 성능
gradient norm	1.0	폭발: 원인 먼저 확인	gradient·NaN

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

PPO 조정 ② 탐색과 가치함수

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
entropy 계수	0.005	탐색 부족: 0.01 비교	행동 std·충돌
초기 log_std	−0.7	포화 많음: −1 비교	clamp 비율
value loss 계수	0.5	가치 오차 먼저 분석	value loss·설명력
관측 정규화	목표 clamp ±1	분포·단위 먼저 확인	입력 포화 비율

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

PPO 조정 ③ 시간과 표본

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
gamma	0.99	먼 성과: 0.995 비교	지연 보상·분산
GAE lambda	0.95	분산 큼: 0.9 비교	advantage 분산
rollout 길이	64 step	긴 의존성: 128 비교	메모리·표본 수
minibatch	256	총 표본 수와 맞춤	업데이트 횟수

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

PPO 조정 ④ 숫자보다 먼저 볼 것

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
timeout bootstrap	현재 terminal 처리	truncated 별도 처리	value 편향
보상 스케일 변경	value target도 변경	학습 안정성 재확인	value loss · KL
이어 학습	optimizer 상태 복원	설정 변경 반영 확인	실제 lr 출력
좋은 정책 선택	마지막 파일 아님	고정 평가로 선택	완주 · 충돌 · 시간

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

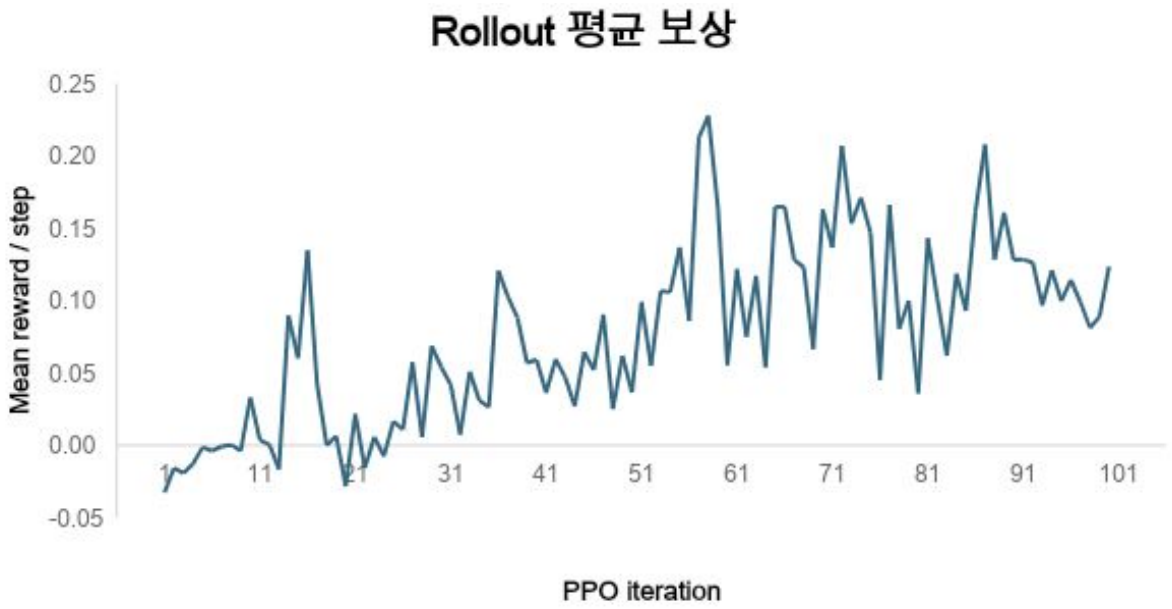
학습 실행 명령

```
export OMNI_KIT_ACCEPT_EULA=YES
PY=~/IsaacLab/.venv/bin/python
mkdir -p runs

$PY scripts/lab_python.py rl/train_ppo.py ₩
  --iterations 100 --num-envs 8 --seed 7 ₩
  --output runs/maze_ppo --visualizer none ₩
  > runs/train.log 2>&1

tail -f runs/train.log
```

실제 학습 곡선

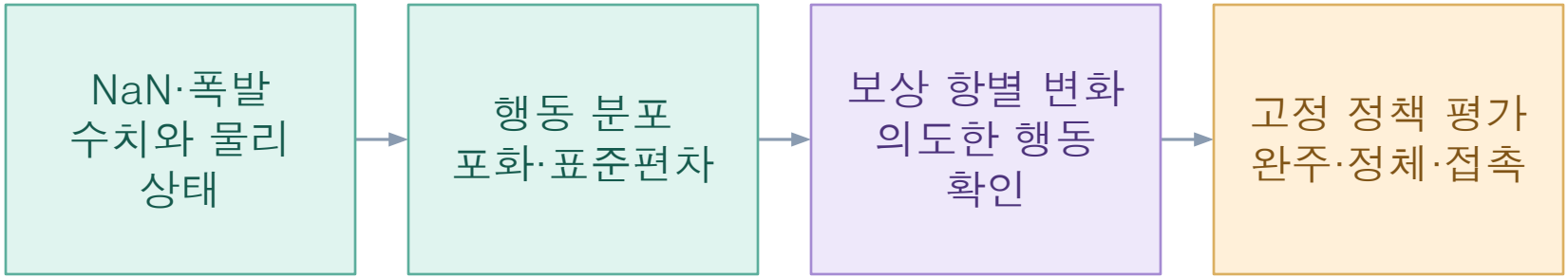


51,200 transition

8개 병렬 환경

완주 성공률은 별도 평가

로그를 읽는 순서



짧은 시연 영상과 수치 로그를 같은 체크포인트로 묶는다.

체크포인트 저장과 이어 학습

```
runs/maze_ppo/policy.pt
runs/maze_ppo/training.json

$PY scripts/lab_python.py rl/train_ppo.py ₩
  --resume runs/maze_ppo/policy.pt --iterations 300 ₩
  --output runs/maze_ppo_extended --visualizer none

sha256sum runs/maze_ppo_extended/policy.pt
```

최신 파일보다 고정 조건 평가 결과를 기준으로 정책을 선택한다.

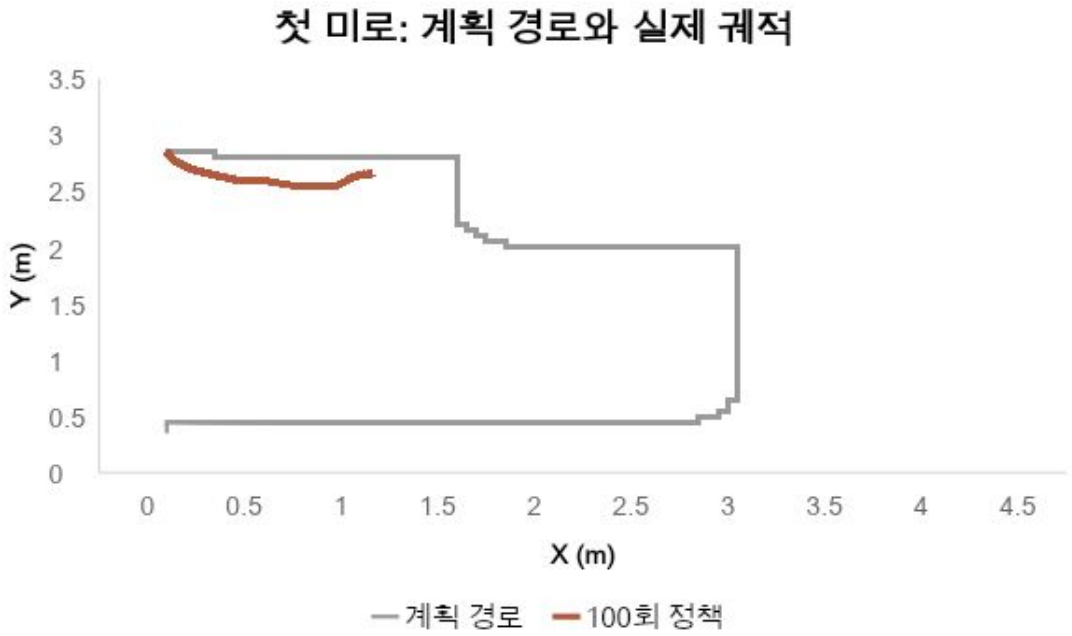
정책 평가 실행

```
$PY scripts/lab_python.py rl/evaluate.py ₩
  --checkpoint runs/maze_ppo/policy.pt ₩
  --output evidence/maze-eval --visualizer none

cat evidence/maze-eval/report.json
head evidence/maze-eval/trajectory.csv
```

평가 시작 위치는 한 번만 설정한다.
주행 중 포즈는 PhysX가 계산한다.

첫 정책의 주행 결과



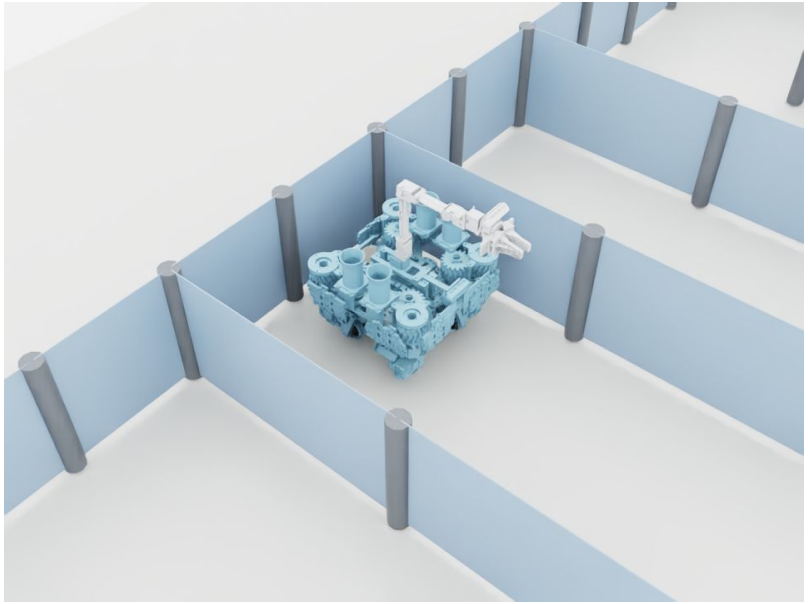
완주 실패

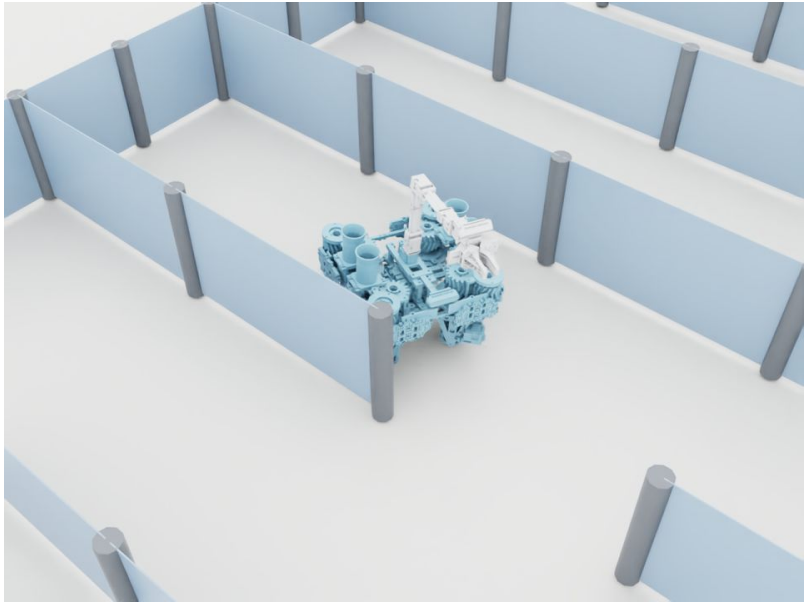
160초 평가

최종 목표 오차
2.53 m

중간 구간 정체

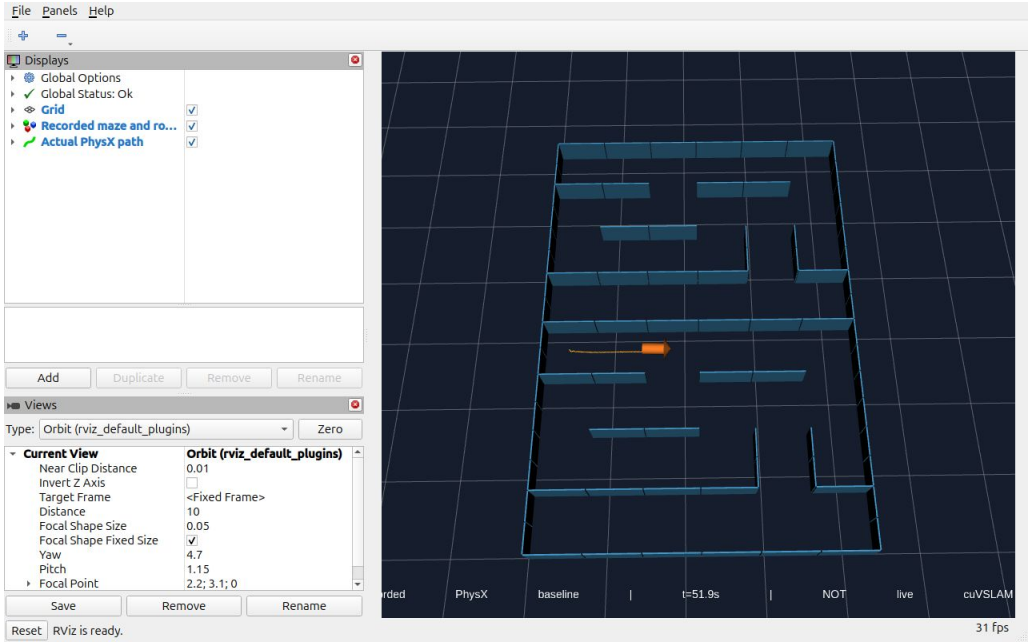
시뮬레이션 주행 화면





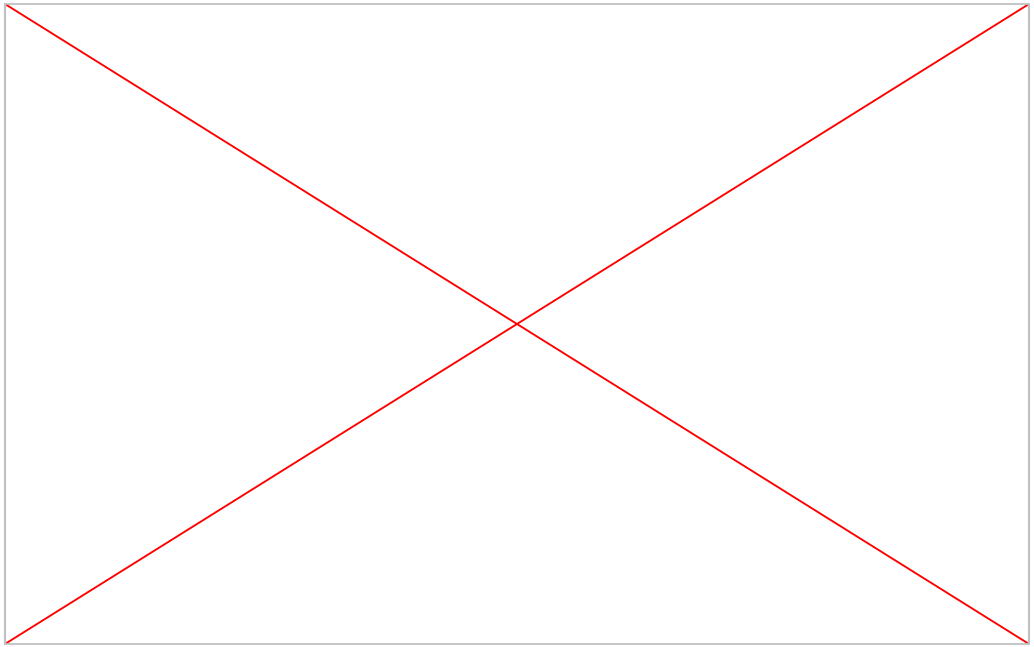
실제 PhysX 포즈를 Isaac Sim으로 재생 렌더링한 화면 / 100회 정책

RViz2 GUI: 기록 궤적 시각화



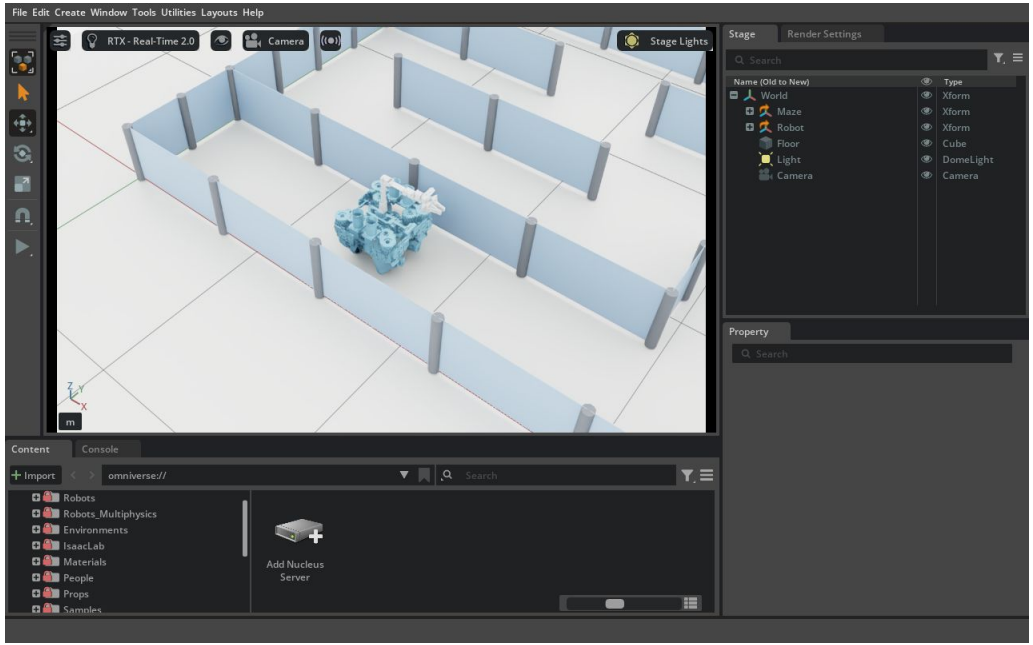
실제 RViz2 창 캡처 / 미로·로봇 화살표·측정 경로 / 실시간 cuVSLAM 결과 아님

RViz2 GUI: 20초 화면 녹화



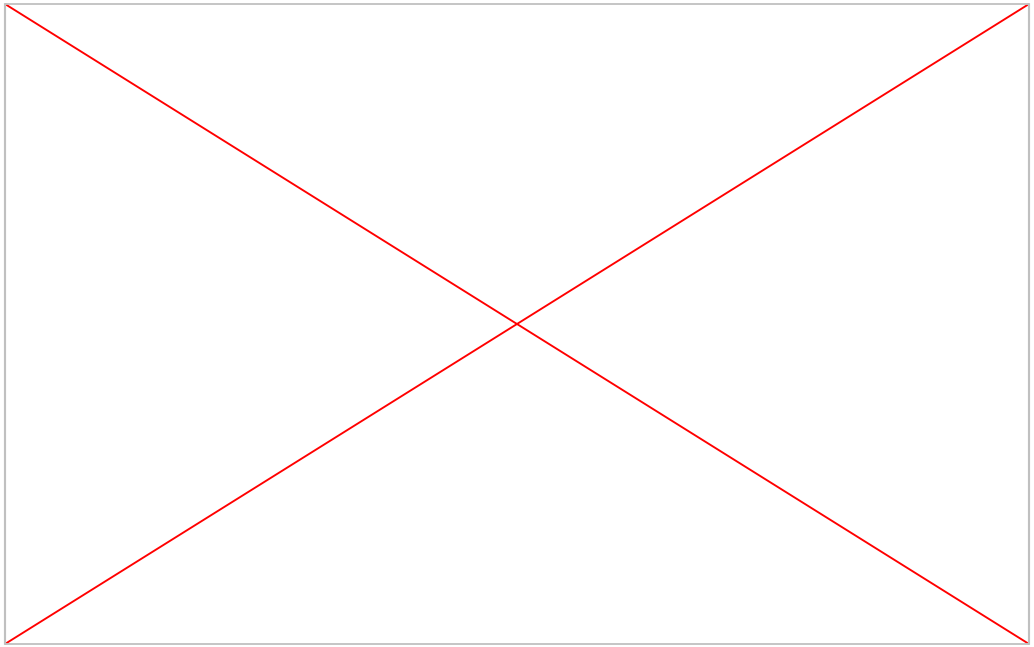
▶ 클릭 재생 / 실제 RViz2 창 녹화 / 기록 궤적을 2배속으로 시각화

Isaac Sim GUI: 미로와 로봇



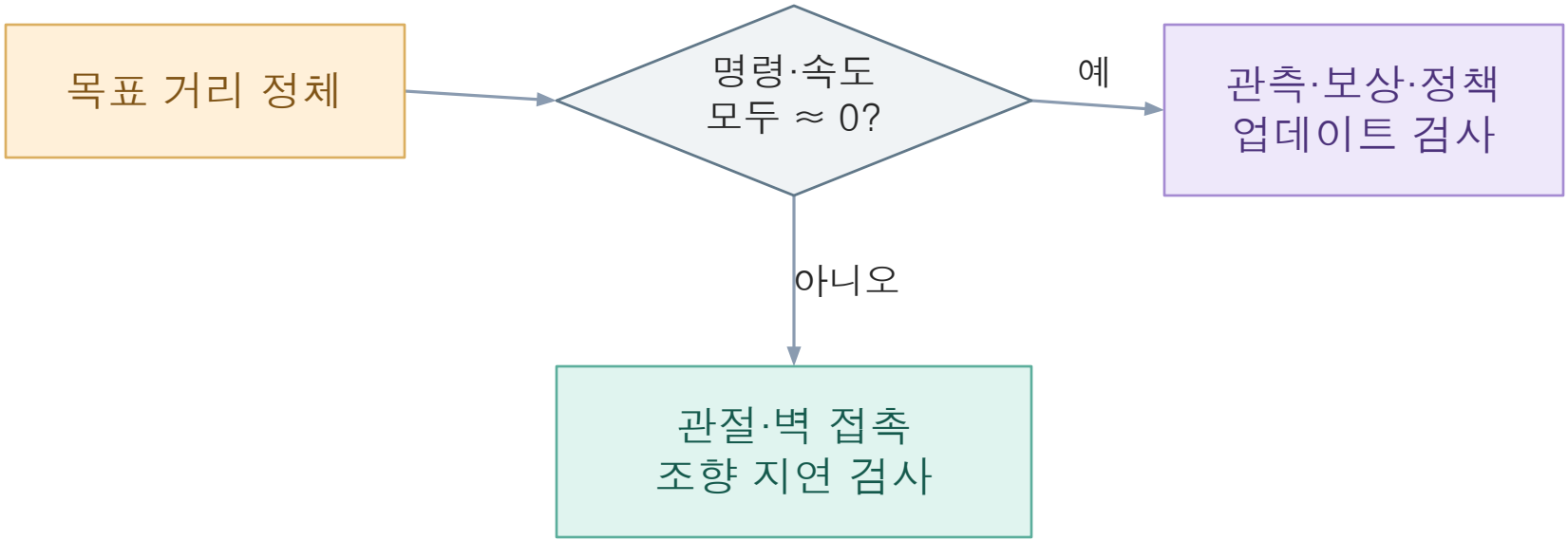
실제 Isaac Sim 창 캡처 / 완주한 비교 제어기의 PhysX 포즈 기록 재생

Isaac Sim GUI: 20초 화면 녹화



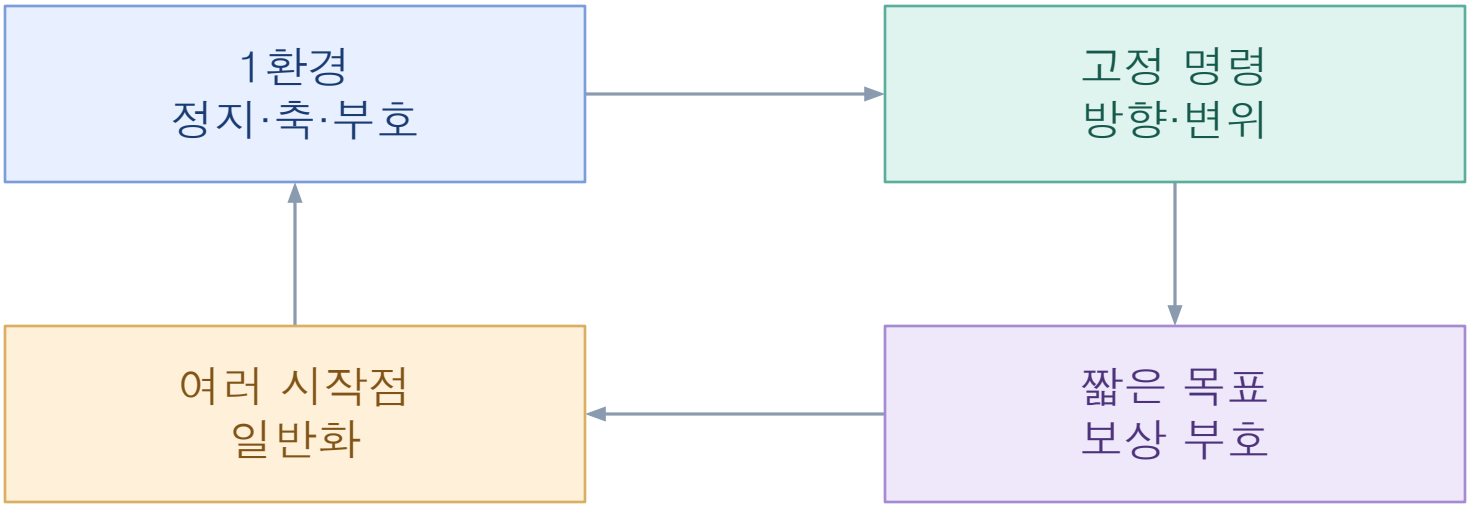
▶ 클릭 재생 / 실제 Isaac Sim 창 녹화 / 비교 제어기 실행 기록을 2배속 재생

정체 현상의 진단 흐름



한 원인씩 수정한 뒤 같은 체크포인트 평가 조건으로 비교한다.

학습이 안 될 때 점검 순서



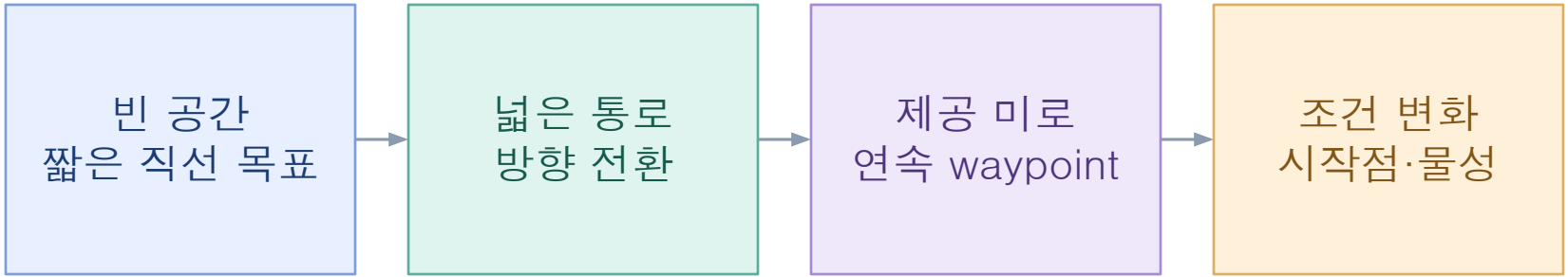
순서: 모델 문제, 환경 문제, 알고리즘 문제를 차례로 좁힌다.

보상 설계 실수와 수정

관찰한 현상	우선 확인할 내용
정지 상태를 유지	진행률 보상과 시간비용 크기
벽에 계속 밀착	접촉·벽 거리 관측과 목표 위치
목표 주변에서 진동	도달 기준과 감속 처리
리셋 순간 보상 급증	새 목표의 이전 거리 초기화

원인 가설을 하나씩 바꾸고 같은 조건에서 재평가한다.

커리큘럼 설계



처음부터 복잡한 미로 전체 성공 보상 하나만 주면 탐색이 어려워진다.

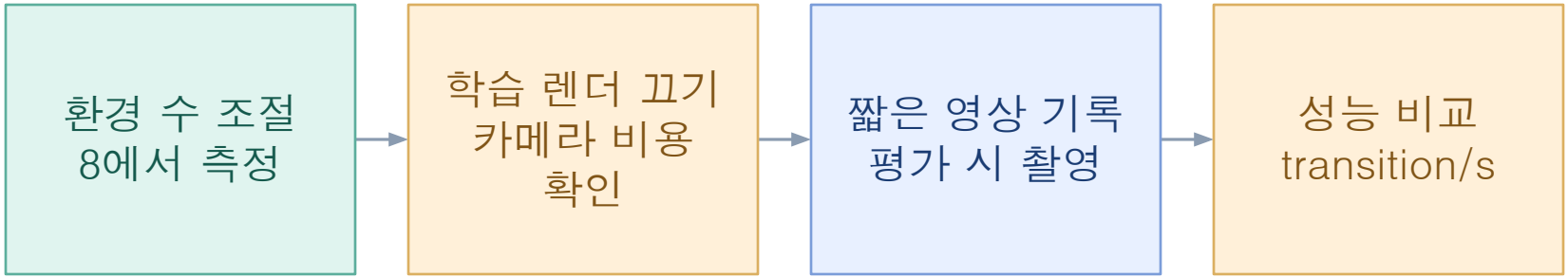
KANT

학습 환경 조정 사례

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
학습 목표 거리	경로 ±12점	실패 많음: ±6점	도달률·방향별 차이
시작 위치 오차	XY ±0.05 m	안정 후 단계 확대	벽 안에서 reset?
episode 길이	8초	도달 가능한 시간 확보	timeout 비중
병렬 환경	8개	메모리 보며 16개	표본/s·재현성

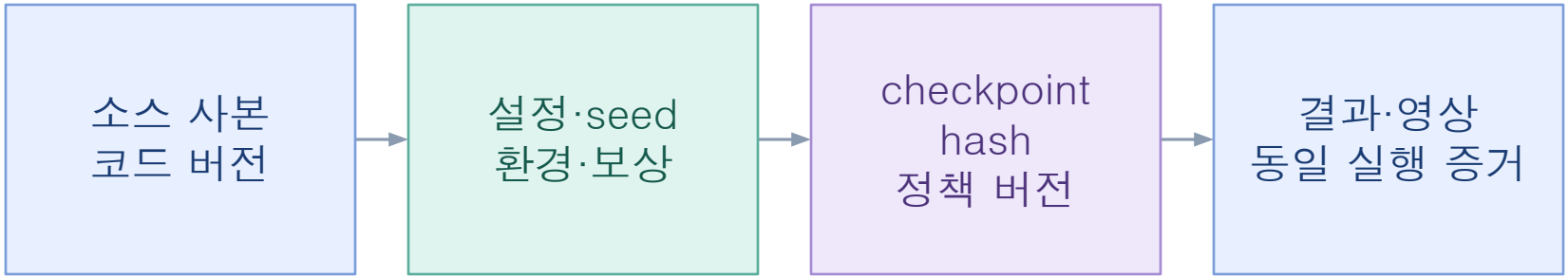
제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

GPU 메모리와 학습 속도 팁



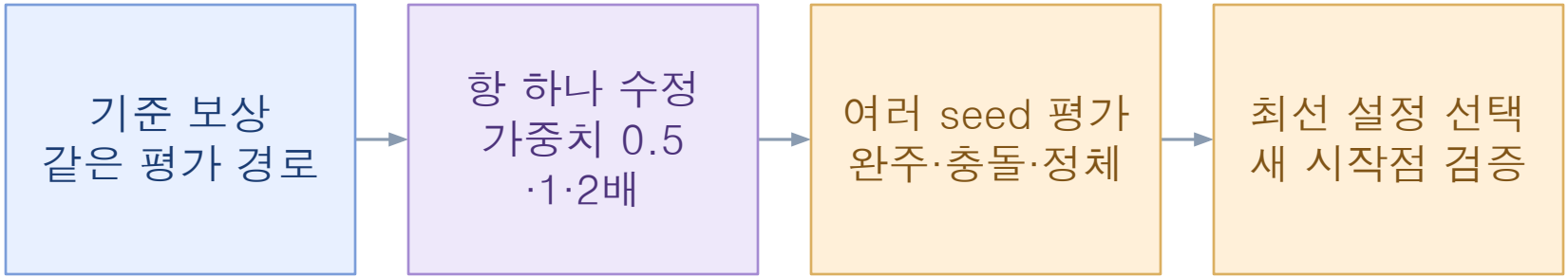
환경 수 증가는 GPU 사용량만이 아니라 CPU·메모리 병목도 함께 측정한다.

재현 가능한 실험 기록



학습 횟수만 적은 결과는 다른 팀이 재현하기 어렵다.

보상 가중치 실험 순서



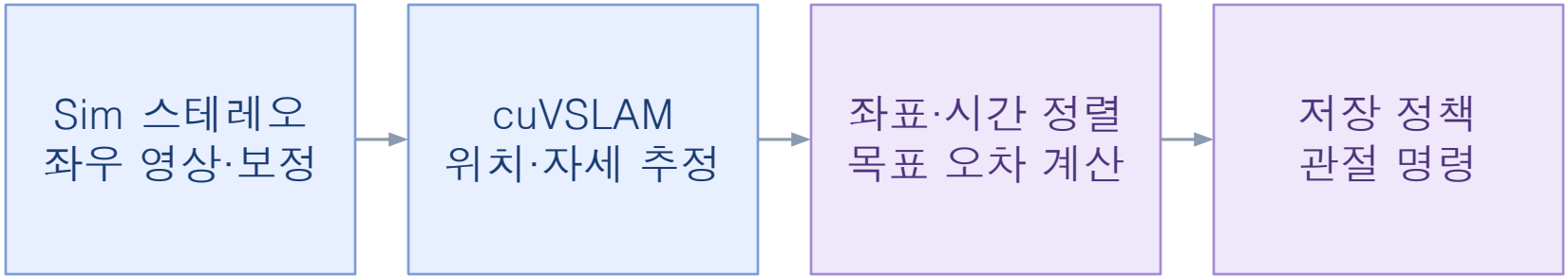
최고 학습 보상보다 완주율·충돌·소요시간으로 설정을 선택한다.

실험표를 작성하는 예시

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
A: 기준	진행 4 / 시간 .02	나머지 고정	성공·충돌·시간
B: 진행만 변경	진행 2	A와 같은 seed	정체 개선 여부
C: 시간만 변경	시간 .01	A에서 다시 시작	충돌 감소 여부
D: 최종 후보	우수 설정 하나	새 시작점·seed	일반화 성능

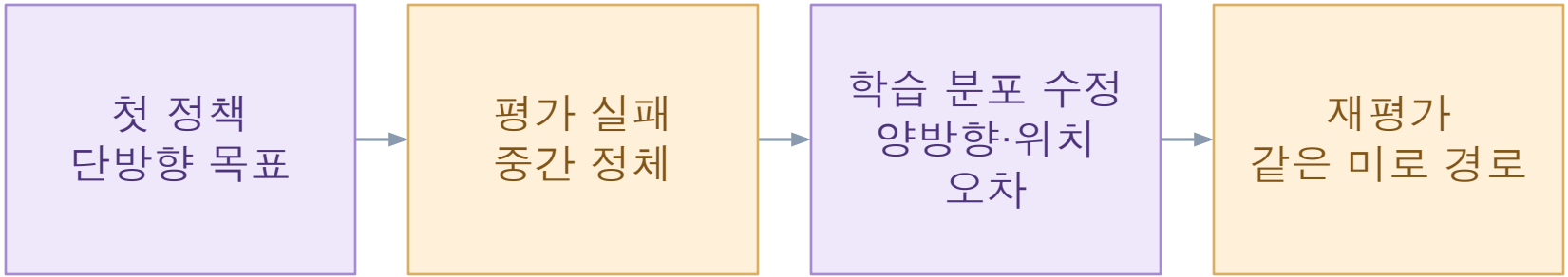
제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

SLAM과 정책 연결 구조



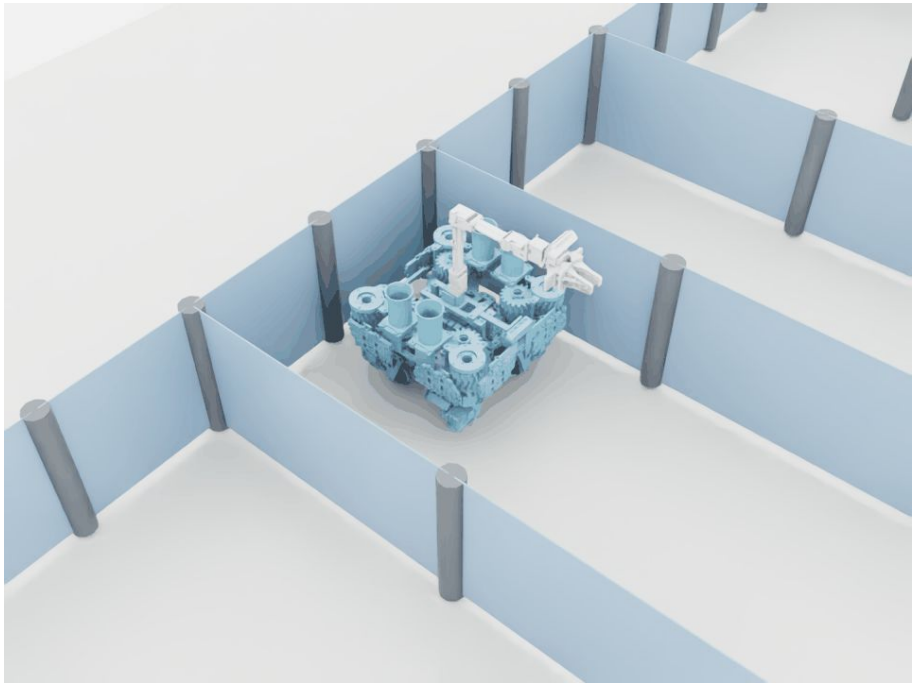
추정 pose를 사용할 때 지연·노이즈·tracking loss에 대한 평가가 추가된다.

시연 실험: 학습 조건 수정



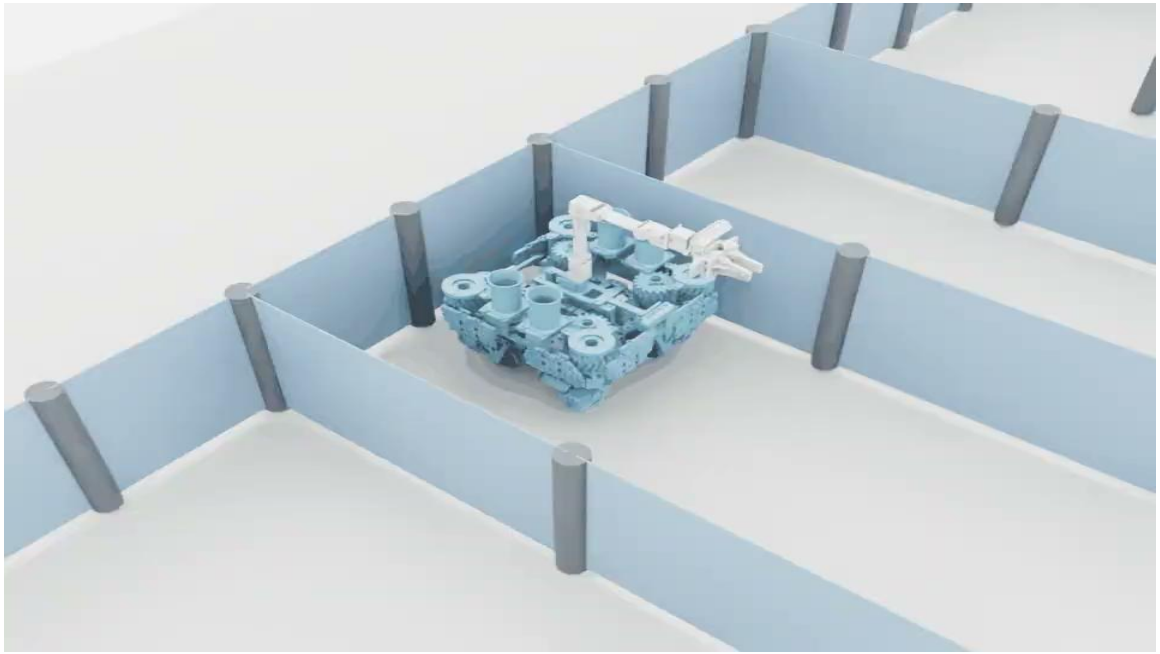
조향 분기 유지도 별도로 검증한다. 각 변경의 결과를 실행 기록에 남긴다.

시연: 주행 GIF와 재생 방법



GIF는 슬라이드쇼에서 확인 / 0~40초를 약 7배속 재생 / 다음 장: MP4 재생

시연: 실제 기록 MP4 재생



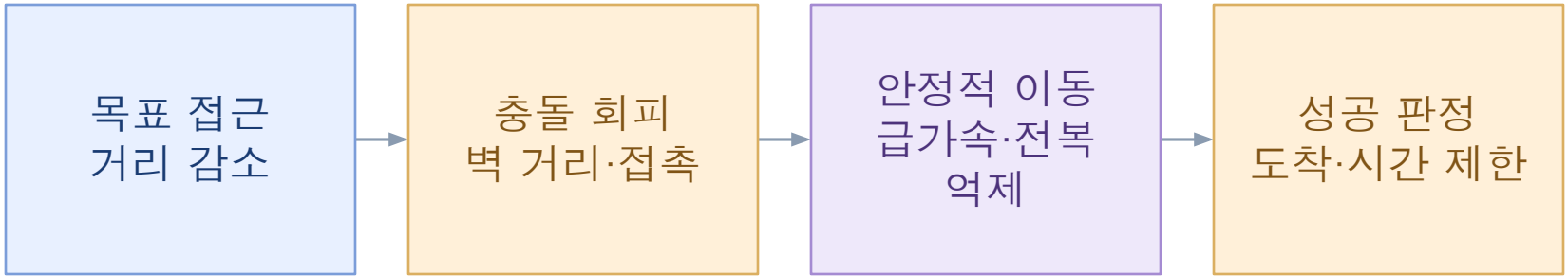
▶ 클릭하여 재생 / 실제 PhysX 포즈 기록 재생 / 36개 표본 프레임을 약 40초로 재생

실행 결과 비교

실행	완주	모의 시간 (s)	목표 오차 (m)
PPO 100회	실패	160.0	2.525
PPO 316회	실패	73.0	2.068
비교 제어기	성공	110.2	0.118
PPO 546회	실패	52.5	2.728

각 1회 평가 / 알려진 경로·Sim 위치 사용 / 비교 제어기는 강화학습 정책과 구분

보상 설계: 원하는 행동부터 정의



먼저 행동과 측정값을 정한 뒤 보상 항과 가중치를 정한다.

보상 크기를 숫자로 비교하기

현재 구현: $r = 4 \times (d_{prev} - d) / dt - 0.02$
$- 0.01 \times \|a\|^2 + 2 \times I$[도달]

dt = 1/15 s, 1 cm 접근 → 진행 보상 +0.60
1 cm 후퇴 → −0.60 / 정지·행동 0 → −0.02

$\|a\|^2 \leq 2$ → 행동 비용은 최대 −0.02 / step

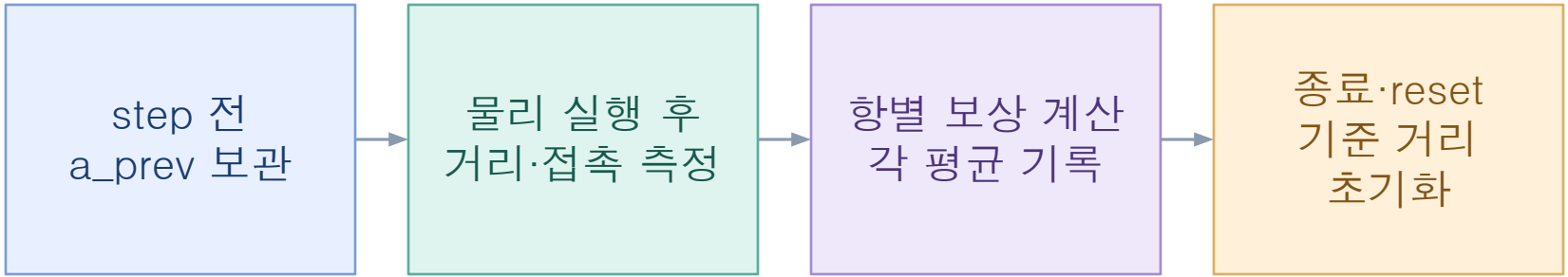
충돌·부드러움 항을 추가하는 법

$$r = r_{base} - w_{hit} \times I[\text{충돌}] - w_{near} \times \max(0, d_{safe} - d_{wall})^2 - w_{smooth} \times \|a_t - a_{prev}\|^2$$

d_wall: 로봇 외곽부터 벽까지의 거리
충돌: 바닥 접촉을 제외한 차체·벽 접촉

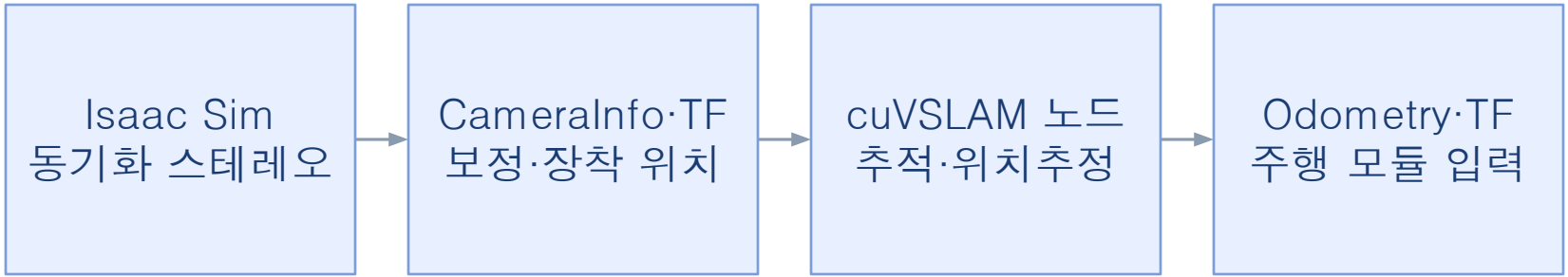
추가 제안: 센서와 항별 로그를 구현한 뒤 재학습

보상 구현에서 놓치기 쉬운 상태



보상 항의 부호·단위·스케일을 작은 수치 예제로 먼저 검증한다.

cuVSLAM 입력과 출력



cuVSLAM의 위치추정 결과와 장애물 지도는 서로 다른 데이터다.

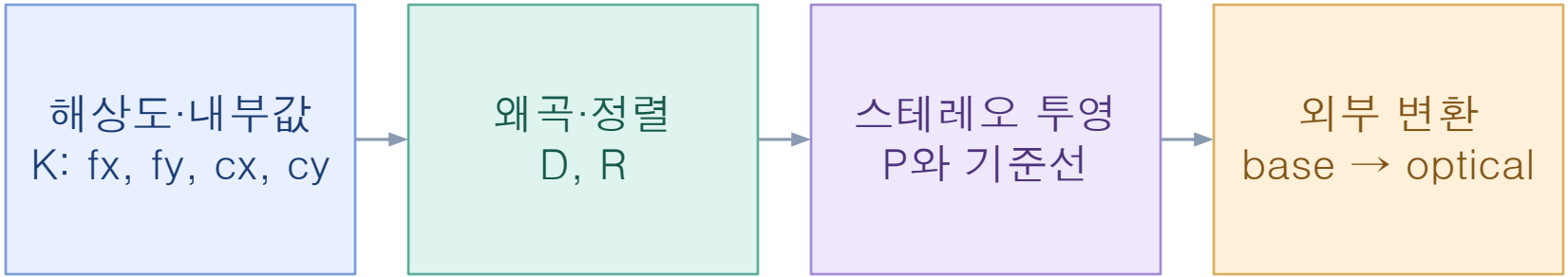
cuVSLAM 필수 파라미터

```
use_sim_time: true
rectified_images: true
enable_imu_fusion: false

base_frame: base_link
odom_frame: odom
map_frame: map

camera_optical_frames:
  [sim_left_optical, sim_right_optical]
```

카메라 보정: 무엇을 맞추나



카메라를 옮기거나 해상도를 바꾸면 CameraInfo와 TF도 함께 갱신한다.

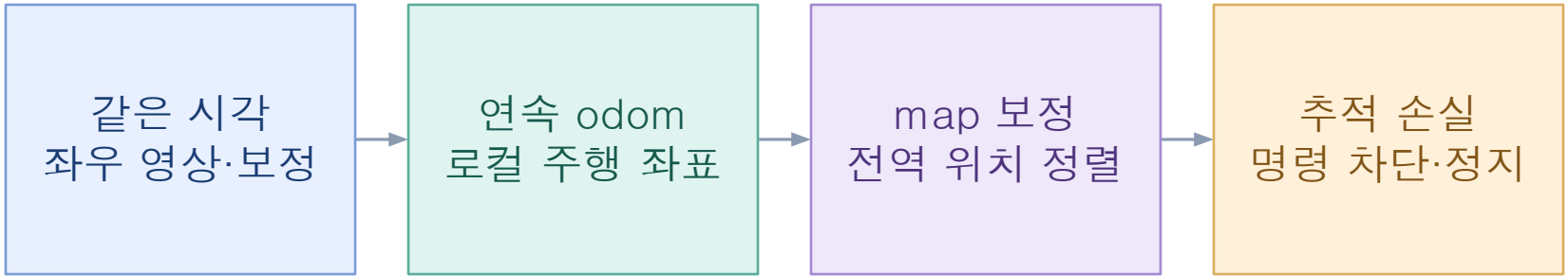
cuVSLAM 토픽 연결과 확인

```
source /opt/ros/lyrical/setup.bash
ros2 launch ./launch/cuvslam_sim.launch.py

ros2 node info /visual_slam_node
ros2 topic hz /front_stereo_camera/left/image_rect_color
ros2 topic hz /front_stereo_camera/right/image_rect_color

ros2 topic echo /clock --once
ros2 run tf2_ros tf2_echo base_link sim_left_optical
```

시간·좌표·추적 손실 점검



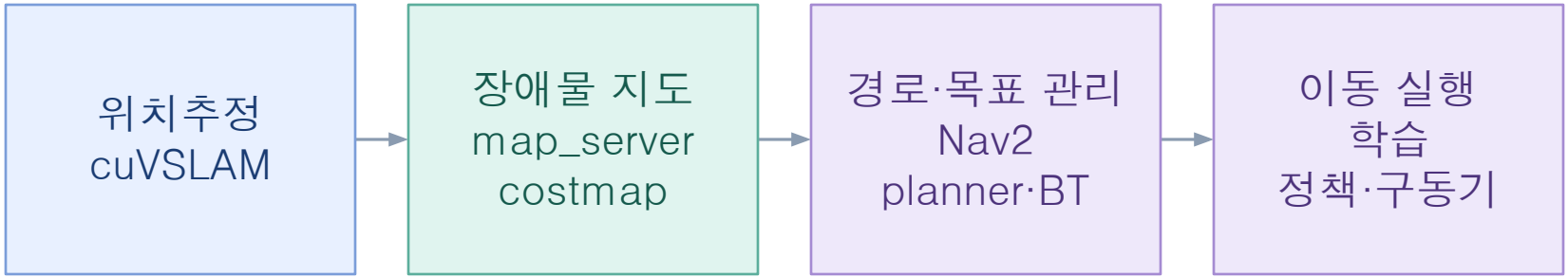
원점·축·회전·지연을 검증한 뒤 Sim 정답 위치를 추정 위치로 교체한다.

cuVSLAM: 조정과 수정의 구분

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
영상이 뒤틀림	K·D·R·P	보정값 수정	수평 에피폴라
추적이 끊김	노출·무늬·동기	입력 품질부터	추적률·지연
위치 축이 틀림	optical TF	축·외부보정 수정	직진·회전 방향
IMU를 추가함	fusion=false	시간·외부보정 후 on	드리프트·안정성

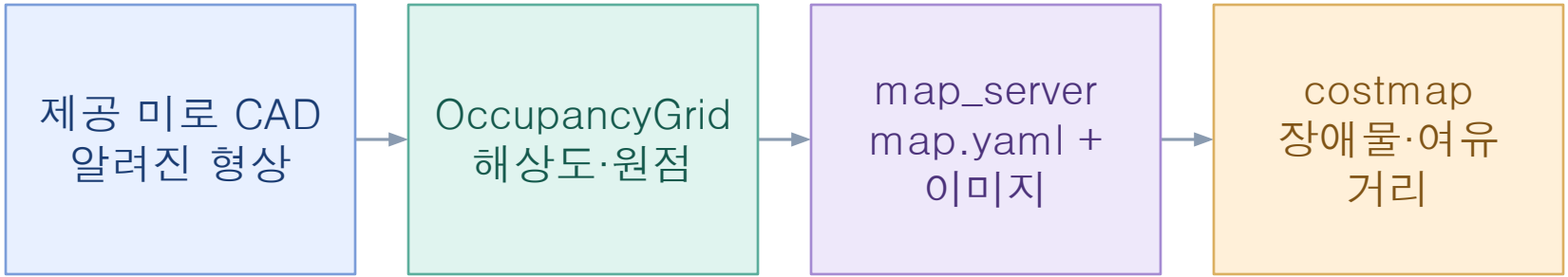
제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

자율주행 패키지 역할 분담



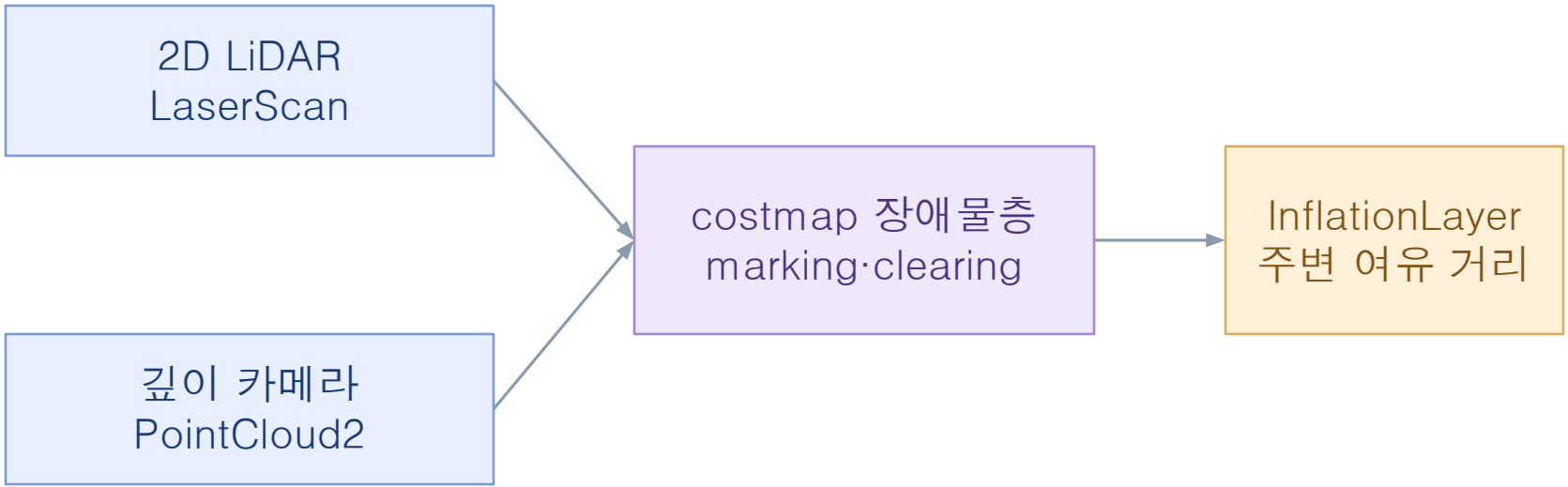
경로계획, 위치추정, 모터 제어를 하나의 패키지가 모두 해결하지 않는다.

자율주행에 필요한 지도 만들기



미지 환경 확장: 깊이 센서 + nvblox 또는 2D LiDAR + slam_toolbox.

장애물 센서는 무엇을 쓰나



센서 대안 중 하나를 선택하거나 함께 사용한다.

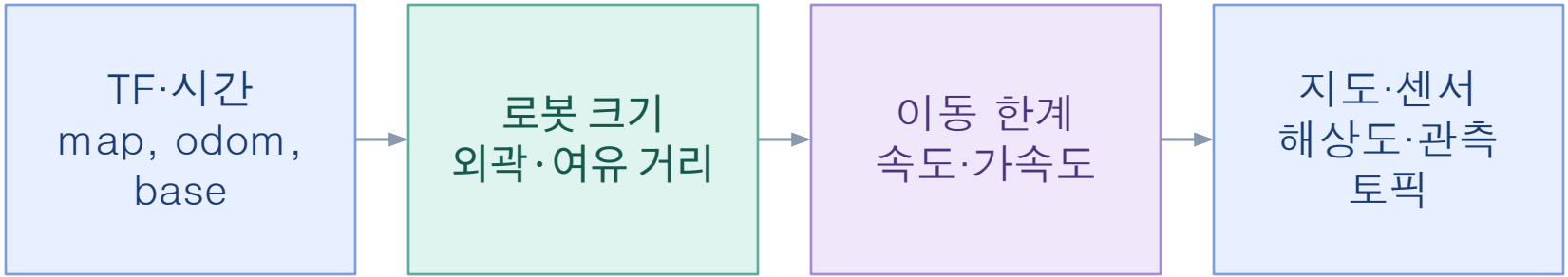
Nav2 설치와 설정 파일 준비

```
sudo apt install ros-lyrical-navigation2 ₩
  ros-lyrical-nav2-bringup

source /opt/ros/lyrical/setup.bash
NAV2_SHARE=$(ros2 pkg prefix nav2_bringup)/share/nav2_bringup

mkdir -p config
cp "$NAV2_SHARE/params/nav2_params.yaml" config/nav2.yaml
```

Nav2에서 팀별로 바꿀 설정



M38 치수·횡이동 제한을 다른 팀 로봇에 그대로 적용하지 않는다.

KANT

Nav2: 통로·속도·도착 조정

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
좁은 통로 실패	footprint·padding	실제 외곽부터 수정	실제 충돌 여유
벽 가까이 주행	inflation 설정	비용 분포 점검	여유 거리·통과
코너를 크게 벗어남	속도·가속도	우선 저속으로	추종오차·제동거리
목표에서 진동	goal tolerance	센서 오차보다 여유	정지·방향 오차

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

Nav2 기본 제어기로 먼저 확인

```
planner: nav2_navfn_planner::NavfnPlanner
controller: nav2_mppi_controller::MPPIController

FollowPath:
  plugin: "nav2_mppi_controller::MPPIController"
  motion_model: "omni"
  omni:
    plugin: "mppi::OmniMotionModel"
```

발췌 예시: 실제 설치 버전의 기본 설정과 병합

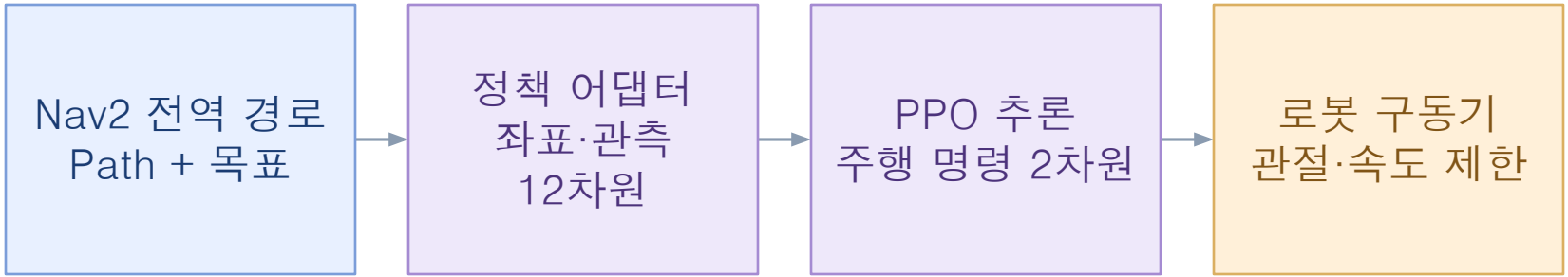
KANT

Nav2: MPPI 파라미터를 바꿀 때

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
motion_model	스워브: Omni	실제 구동 구조와 일치	횡이동 가능?
vx·vy·wz 제한	구동기 실측값	포화 시 한계 낮춤	명령 대비 실제 속도
model_dt·time_steps	예측 시간 결정	제동·코너 길이 고려	예측 궤적·연산시간
batch_size·온도	샘플·선택 민감도	기본값에서 한 항씩	지연·충돌·진동

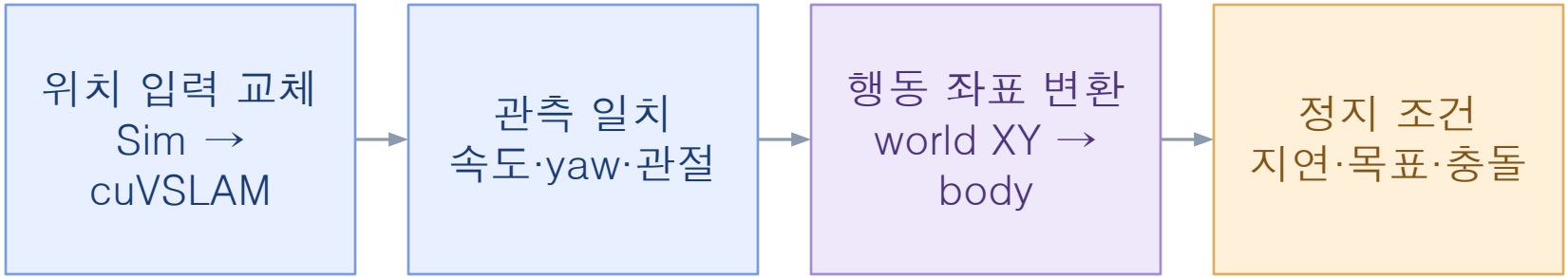
제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

학습 정책을 Nav2에 연결하는 법



MPPI와 PPO를 동시에 cmd_vel에 발행하지 않는다. 실행 모드는 하나를 선택한다.

현재 정책을 연결할 때 남은 작업



현재 2차원 정책은 모든 로봇·모든 Nav2 목표에 바로 적용되는 제어기가 아니다.

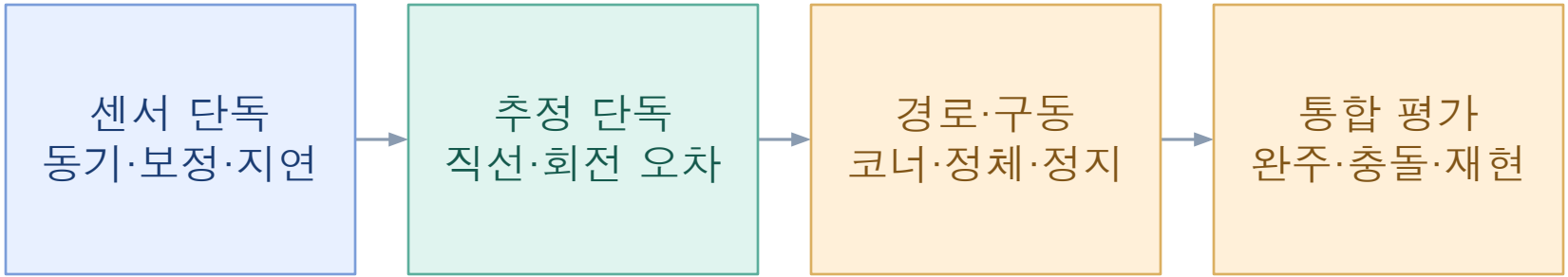
Nav2 실행 순서와 목표 보내기

1. Sim 센서·관절 → TF·clock → cuVSLAM 추적 확인
2. map_server를 시작하고 지도 원점 정렬·활성화

```
ros2 launch nav2_bringup navigation_launch.py ₩
  use_sim_time:=true params_file:=$PWD/config/nav2.yaml
```

3. RViz에서 map·TF·costmap·경로 확인
4. Nav2 Goal 지정 → 실제 변위·도착 결과 확인

자동 주행 검증 체크포인트



패키지가 실행되는 것과 실제 폐루프 자율주행 성공은 각각 확인한다.

costmap 설정 예시: LiDAR 입력

```
obstacle_layer:
  plugin: "nav2_costmap_2d::ObstacleLayer"
  observation_sources: scan

  scan:
    topic: /scan
    data_type: "LaserScan"
    marking: true
    clearing: true
```

local/global_costmap의 plugins 목록에도 등록한다.

정적 지도 서버 활성화

ros2 run nav2_map_server map_server --ros-args ₩
  -p use_sim_time:=true -p yaml_filename:=$PWD/maps/maze.yaml

별도 터미널에서:
ros2 lifecycle set /map_server configure
ros2 lifecycle set /map_server activate

ros2 topic info /map
RViz: 지도 벽과 Sim 좌표·로봇 위치가 일치하는지 확인

부록: 스워브 역기구학 수식

$$u_i = v_x - \omega_z y_i, \qquad v_i = v_y + \omega_z x_i$$

$$\theta_i = \operatorname{atan2}(v_i, u_i) - \beta_i$$

$$\dot{\phi}_i = \sigma_i \frac{\sqrt{u_i^2 + v_i^2}}{r_i}, \qquad \sigma_i \in \{-1, +1\}$$

vx, vy, ωz: 차체 명령 / xi, yi: 모듈 위치 / βi: 영점 / ri: 바퀴 반경

부록: 스워브 모듈 정보를 만드는 법

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
모듈 위치	(x_i, y_i)	base 기준 m	앞뒤·좌우 부호
바퀴 반경	시연 0.030687 m	팀 USD 실측	속도 환산·미끄럼
조향 영점	beta_i	관절 0의 굴림 방향	전진 명령 테스트
관절 순서	조향4 + 바퀴4	이름으로 매핑	동일 모듈 짝

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

부록: 스워브 역기구학 계산

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
모듈 X 속도	$u_i = vx - wz \cdot y_i$	body 좌표 사용	순수 전진
모듈 Y 속도	$v_i = vy + wz \cdot x_i$	rad/s와 m 혼동 금지	순수 횡이동
조향 목표	$\operatorname{atan2}(v_i, u_i) - \text{beta}_i$	각도 wrap	제자리 회전
바퀴 각속도	$\sqrt{u^2+v^2} / r$	회전 부호 적용	모든 모듈 접선

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

KANT

부록: 조향 최적화와 반전 방지

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
조향 ±90° 제한	theta±pi, 속도 반전	각도와 부호 동시 수정	같은 굴림 방향
경계에서 채터링	시연 hold=0.06 rad	작게 조정·재시험	분기 전환 횟수
조향 속도 한계	시연 1 rad/s	실제 구동 한계 반영	목표·실제 지연
정렬 전 바퀴 회전	max(0, cos(error))	정렬 동안 감속	횡미끄럼·정체

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

부록: Isaac Lab 구동기 설정

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
steer 위치 제어	Kp=30, Kd=3	진동: Kp ↓ · Kd 검토	각도 오차 · 토크
wheel 속도 제어	Kp=0, Kd=1	추종 지연: Kd 검토	속도 오차 · 슬립
effort limit	시연 1 N · m	모터 한계로 설정	포화 · 발열 가정
articulation root	base_link	루트 · 충돌체 검증	안정 높이 · 축

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

부록: Isaac Lab 관측·시간 설정

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
정책 주기	60 Hz / decimation4	15 Hz 동작	추론·센서 지연
행동	시연 2: world vx,vy	yaw 제어는 별도	일반 twist와 차이
관측	시연 12차원	추가 시 네트워크 수정	단위·순서 일치
실패 종료	높이·기울기·NaN	로봇 크기에 맞춤	정상 동작 오판

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

부록: 스워브와 Nav2 연결

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
Nav2 출력	body vx, vy, wz	메시지 타입 확인	Twist / Stamped
Omni 모델	횡이동 허용	vy 제한도 지정	역기구학 연결
실제 odom	센서 기반 속도	명령 복사 금지	실제 이동 반영
명령 중재	Nav2 또는 PPO	한 모드만 활성화	만료·손실 시 정지

제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

부록: 스워브 증상별 점검표

상황 / 항목	기준 / 예시	조정 방향	확인할 결과
전진인데 회전	모듈 순서·부호	한 바퀴씩 확인	대칭 속도
횡이동이 대각선	조향 영점·beta	USD 축 다시 측정	vy 단독 변위
코너에서 떨림	분기·조향 지연	저속·각도 로그	반전 횟수
명령은 큰데 정지	접촉·토크·슬립	보상보다 물리 우선	실제 관절·속도

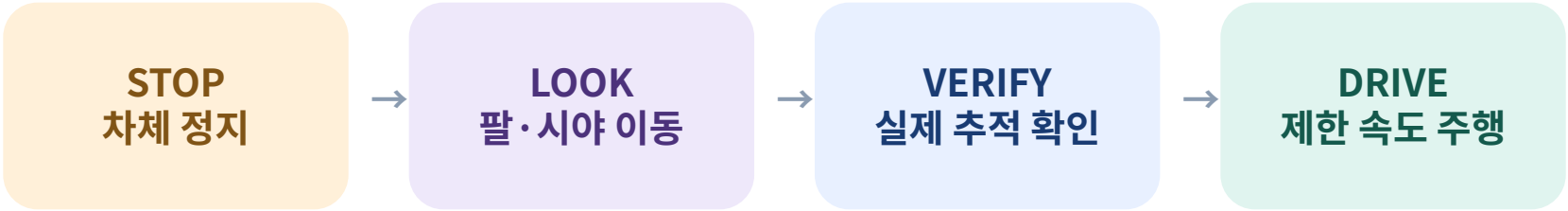
제안값은 실험 시작점이다. 같은 평가 조건에서 한 항씩 비교한다.

팔 카메라: 현재 모델과 목표

상황 / 항목	설계 / 시작점	조정·구현 방법	평가할 결과
팀별 로봇	URDF·USD 준비	장착 TF·관절 범위 확인	시야 도달 가능 영역
M38 시연 모델	팔 5축 / 각 ±20°	실제 사양과 대조	측면·후방 자세 가능?
현재 카메라	차체 고정 stereo	팔 끝 리그로 새 구성	좌우 상대 TF 고정
현재 정책	입력 12 / 행동 2	팔 입력·출력 확장	기존 가중치 호환 불가

확장 설계 · 제안값은 실험 시작점 / 팔 카메라 PPO 학습·실행은 아직 미검증

시야 확보 후 주행: 전체 흐름



추적 끊김 · 방향 변경 · 시야 불가능 → STOP으로 복귀

정렬만으로 추적 성공을 가정하지 않는다. 불확실하면 정지하고 다시 확인한다.

주행 허용 조건과 조정 예시

상황 / 항목	설계 / 시작점	조정·구현 방법	평가할 결과
STOP → LOOK	속도 <0.02 m/s	회전도 <0.03 rad/s	실제 정지 확인
LOOK → VERIFY	시야 오차 <15°	팔 속도 <0.05 rad/s	흔들림·블러
VERIFY → DRIVE	추적 양호 0.5초	0.3·0.5·1초 비교	오출발·대기 시간
DRIVE → STOP	오차 >25° / 추적 실패	지연 >0.15초도 차단	정지 거리·복구율

확장 설계 · 제안값은 실험 시작점 / 팔 카메라 PPO 학습·실행은 아직 미검증

Isaac Lab: 관측·행동 확장

상황 / 항목	설계 / 시작점	조정·구현 방법	평가할 결과
차체 관측	목표·속도·조향	다음 진행 방향 추가	후진·옆 이동 구별
팔 관측	q·q̇·joint limits	카메라 방향·오차 추가	관절 포화·불가능 자세
추적 관측	유효 여부·지연·유지시간	실제 / proxy 채널 분리	학습·실행 입력 일치
행동 출력	차체 2축 + 팔 n축	속도 제한된 Δq 목표	M38 예: 7차원

확장 설계 · 제안값은 실험 시작점 / 팔 카메라 PPO 학습·실행은 아직 미검증

움직이는 카메라 → 차체 위치

$$^{o}T_{b}(t) = {}^{o}T_{r}(t)\left({}^{b}T_{r}(q(t))\right)^{-1}$$

$$^{b}T_{r}(q(t)) = \mathrm{FK}(q(t))\,{}^{e}T_{r}$$

영상 시각 t의 관절 자세 q(t)를 사용한다. 최신 TF로 바꿔 끼우지 않는다.

cuVSLAM: 움직이는 팔의 TF

상황 / 항목	설계 / 시작점	조정·구현 방법	평가할 결과
추정 기준 프레임	base_frame=camera_rig	rig→좌우 camera 고정	rig 자세를 먼저 추정
팔의 동적 변환	base→rig = FK(q(t))	영상 시각에 보간	팔 움직임과 주행 분리
차체 위치 환산	odom→rig × rig→base	추정 pose 변환 어댑터	정지 차체의 가짜 이동
TF 발행 주체	cuVSLAM base TF off	어댑터만 odom→base	TF 다중 부모·루프 없음

확장 설계 · 제안값은 실험 시작점 / 팔 카메라 PPO 학습·실행은 아직 미검증

팔 카메라: 보상 설계 수식

$$r_t = w_p g_t \Delta s_t + w_a \frac{|\theta_{t-1}| - |\theta_t|}{\pi} + w_r I_{\text{ready}}$$

$$- w_b (1 - g_t) \|v_t\| \Delta t - w_q \|\dot{q}_t\|^2 \Delta t$$

$$- w_t \Delta t - w_c I_{\text{collision}} + w_g I_{\text{success}}$$

g: 주행 허용 / Δs: 경로 진행량 / θ: 시야 오차 / ready는 구간당 한 번

팔 카메라 보상 사례 ①

상황 / 항목	설계 / 시작점	조정·구현 방법	평가할 결과
시야 확보 후 진행	+w_p × 경로 진행량	허용 상태에서만 지급	맹목 이동 비율
카메라 방향 정렬	+w_a × 각도 오차 감소	각도는 π로 정규화	정렬 시간·오차
시야 확인 완료	+w_r × 최초 ready	구간당 한 번만 지급	정지·출발 반복 악용
시야 없이 이동	−w_b × 속도 × dt	명령 차단도 별도 적용	실제 잔류 이동 거리

확장 설계 · 제안값은 실험 시작점 / 팔 카메라 PPO 학습·실행은 아직 미검증

팔 카메라 보상 사례 ②

상황 / 항목	설계 / 시작점	조정·구현 방법	평가할 결과
팔을 계속 흔듦	$-w_q \times \Sigma q^2 \times dt$	0.01 → 0.05 비교	블러·정렬 지연
추적을 반복 상실	실패 이벤트 비용	실제 상태 / 대리값 구분	추적 지속률·복구 시간
멈춘 채 끝까지 대기	$-0.02 \times dt$	정체 timeout도 평가	안전 정지·완주 함께
팔·벽 접촉 / 도착	충돌 −5 / 성공 +10	최초 이벤트·종료 처리	충돌률·완주율

확장 설계 · 제안값은 실험 시작점 / 팔 카메라 PPO 학습·실행은 아직 미검증

단계별 학습과 승급 조건

상황 / 항목	설계 / 시작점	조정·구현 방법	평가할 결과
1. 정지 상태 정렬	차체 고정 / 팔만 학습	도달 가능한 방향 샘플	정렬률·관절/벽 충돌
2. 확보 후 짧게 이동	직선·측면·후진 분리	0.2 m → 거리 확대	맹목 주행·도착률
3. 미로 방향 전환	코너·막다른 길 포함	차단·재확인 학습	정체·회복 시간
4. 실제 추적 검증	단일 Sim + cuVSLAM	노출·무늬·지연 변화	추적률·차체 오차

확장 설계 · 제안값은 실험 시작점 / 팔 카메라 PPO 학습·실행은 아직 미검증

팔 카메라 PPO 조정 사례

상황 / 항목	설계 / 시작점	조정·구현 방법	평가할 결과
정렬을 기다리지 못함	episode 8 → 16초	정렬 소요시간부터 측정	timeout 비율
팔이 급하게 흔들림	팔 action 범위 축소	lr 3e−4 → 1e−4 비교	속도·KL·추적률
긴 대기 뒤 성과	rollout 64 → 128	gamma .99 → .995 별도	advantage·value 오차
안전 정지만 학습	단계 1부터 재학습	ready 반복보상 제거	정렬률·주행 성공률

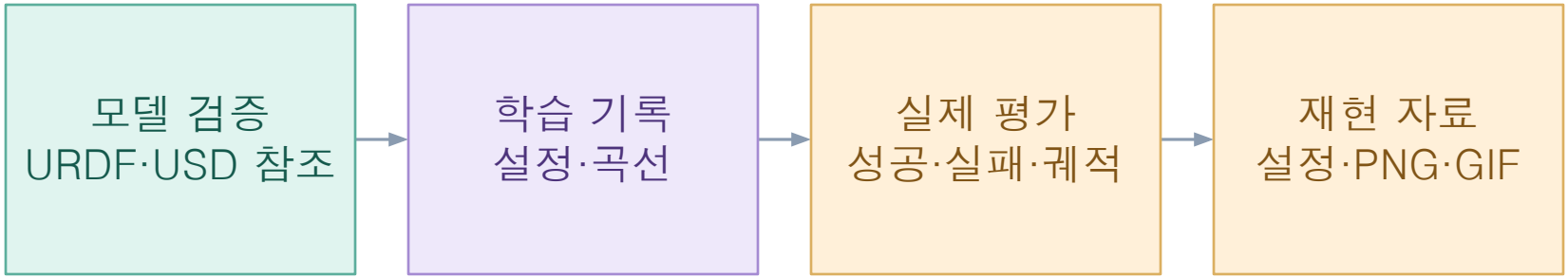
확장 설계 · 제안값은 실험 시작점 / 팔 카메라 PPO 학습·실행은 아직 미검증

실제 검증: 성공 판정표

상황 / 항목	설계 / 시작점	조정·구현 방법	평가할 결과
차체 정지 + 팔만 회전	보정 전/후 odom 비교	GT 차체와 오차 기록	가짜 차체 이동 감소
전·후·좌·우 주행	시야·추적 먼저 확보	모든 방향 같은 거리	방향별 성공·정지 거리
추적 상실·지연 주입	명령 차단 / 회복 후 출발	시각·속도·state 기록	오출발 0 여부
팔이 볼 수 없는 방향	불가능 판정 후 정지	회전/경로 대안 평가	관절 끝 밀기 방지

확장 설계 · 제안값은 실험 시작점 / 팔 카메라 PPO 학습·실행은 아직 미검증

팀별 검증과 제출



모든 팀은 자기 로봇으로 같은 검증 절차를 실행한다.